

分类号_____

学校代码 10487

学号 M201676154

密级_____

华中科技大学

硕士学位论文

异构数据库迁移中间件的设计与实现

学位申请人：黄贤旭

学 科 专 业：软件工程

指 导 教 师：方少红 副教授

答 辩 日 期：2018.12.17

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering**

**Design and Implementation of the Migration
Middleware of Heterogeneous Database**

Candidate : Huang Xianxu

Major : Software Engineering

Supervisor : Assoc. Prof. Fang Shaohong

Huazhong University of Science & Technology

Wuhan 430074, P. R. China

December, 2018

摘要

数据迁移，又称之为数据移植，是指从一种数据环境（源数据库）中提取数据，根据需要迁移或移植到另一种数据环境（目标数据库）中；目前数据迁移除了基本的数据复制功能，还需要实现数据订阅功能以及基于数据迁移与数据订阅功能的多样化需求。

当前研究设计的数据迁移中间件，主要包含数据迁移与数据订阅两大功能模块，数据迁移模块又主要分为全量迁移（将源端数据库所有数据进行迁移）与增量迁移（将源端数据库产生的数据更新在目标端进行回放）。其中全量迁移能够实现数据在异构数据库之间的高速复制，完全能够满足现有的数据异地备灾等功能需求；增量迁移能够实现数据实时同步，主要应用于数据的不停服迁移等市场需求，但是随着互联网产业的发展，对于增量迁移的实时性提出了更高的要求，从而衍生出了数据的异地多活等更复杂的增量迁移需求；数据订阅功能指的是将数据信息封装成消息存放在一个消息队列中，让消费者可以自由消费消息队列中的消息，从而实现生产者与消费者的解耦，提高系统工作效率，主要可以应用在目前的异步消息通知、应用缓存更新等方面。

本文实现的异构数据库迁移中间件也同时实现了这三个主要的功能，包括全量数据迁移，增量数据迁移，数据订阅；能够满足类似不停服数据迁移、数据异地灾备、异地多活(单元化)、跨境数据同步、实时数据仓库、查询报表分流、缓存更新、异步消息通知等功能。

关键词：数据迁移 数据订阅 全量迁移 增量迁移

Abstract

Data migration, refers to extracting data from a data environment (source database), migrating to another data environment (target database) as needed. Currently data migration needs to implement data subscription capabilities and diverse needs based on data migration and data subscription capabilities in addition to move data from a database to another.

The data migration middleware currently researched and designed mainly includes two major functional modules: data migration and data subscription. The data migration module is mainly divided into full migration (migrating all data from the source database) and incremental migration (update the data generated by the source database to the target for playback). The full-scale migration can realize high-speed replication of data between heterogeneous databases, which can fully meet the functional requirements of existing data off-site disaster recovery; incremental migration can realize real-time data synchronization, and is mainly applied to markets such as data non-stop migration, but with the development of the Internet industry, higher requirements are imposed on the real-time nature of incremental migration which leads to more complex incremental migration requirements such as remote data sharing. Data subscription function refers to encapsulate data information into messages in a message queue so that consumers can freely consume the messages in the message queue, with this function we can improve the system work efficiency and decouple between the producer and the consumer and mainly applicable to the asynchronous message notification and Cache updates and more.

The heterogeneous database migration middleware implemented in this paper also implements these three main functions, including full data migration, incremental data migration, and data subscription so that can achieve these functions such as non-stop data migration, data offsite disaster recovery, and live offsite, cross-border data synchronization, real-time data warehouse, query report offload, cache update, asynchronous message notification and more.

Key words : Data migration Data subscription Full migration Incremental migration

目 录

摘 要.....	I
Abstract.....	II
1 绪论	
1.1 研究背景.....	(1)
1.2 研究的意义.....	(2)
1.3 国内研究现状	(3)
1.4 国外研究现状	(4)
1.5 本文概括.....	(5)
2 关键技术	
2.1 Zookeeper 服务	(6)
2.2 Kafka 消息队列.....	(8)
2.3 RPC 通信协议.....	(10)
2.4 Hadoop 框架.....	(10)
2.5 本章小结.....	(12)
3 中间件分析与设计	
3.1 需求分析.....	(13)
3.2 总体设计与任务属性	(16)
3.3 Center 控制中心设计.....	(20)
3.4 Engine 工作节点设计	(23)
3.5 Applier 模块设计	(35)
3.6 数据校验设计	(36)
3.7 结构迁移设计	(38)
3.8 本章小结.....	(41)

4 中间件实现

4.1 开发运行环境	(42)
4.2 Center 控制中心实现	(42)
4.3 Engine 工作节点实现	(44)
4.4 结构迁移模块实现	(49)
4.5 Applier 模块实现	(52)
4.6 数据校验模块实现	(53)
4.7 数据迁移速度测试	(55)
4.8 本章小结	(55)

5 总结与展望

5.1 全文总结	(56)
5.2 展望	(56)

致 谢	(58)
-----------	------

参考文献	(59)
------------	------

1 绪论

1.1 研究背景

随着信息技术的迅猛发展以及信息数据量的海量增长,大多数企业都开始借助多样化的软件来减轻业务压力、缩小企业成本进而提升企业效益,但是随着软件应用愈加广泛以及市场需求的多样性,软件需要不停地推出新的解决方案来解决市场提出的问题,“中间件”的概念便应运而生^[1]。从古老的 C/S 三层架构到现在被大多数互联网公司广泛应用的分布式系统,不同系统间的通信与数据传递就成了瓶颈,而中间件不仅仅解决了硬件差异与通信方式、协议不同的问题,甚至可以适配不同操作系统,使得上层应用的开发和维护成本被大大降低,企业效益大大提升^[2]。

互联网飞速发展的衍生物是软件业务规模的海量增长以及与日俱增的数据压力,随着互联网高速发展、越来越多高新技术逐渐融入寻常百姓的生活,数据量也进入了爆炸式增长阶段,由于这些数据的存在会对社会经济发展以及国家治安管理都产生重大影响,世界各国都把推进经济数字化作为实现创新发展的重要衡量标准^[3]。进而更加推动了全球互联网产业与技术的发展并导致互联网应用数量呈现“井喷”式增长,花样繁多的互联网应用会产生海量的异构数据,同时由于新兴的“互联网+”概念的火热传播,各传统企业也纷纷将各自业务与互联网挂钩,利用智能传感设备或者移动设备大量采集用户行为数据,进而衍生出与人们生活工作联系更加紧密的云办公、云购物等极大改变人们生活方式的智能服务,而这些服务产生的数据被称为非关系型数据^[4]。

非关系型数据库的发展与近年来互联网产业的飞速发展脱不开关系,随着数据量愈加庞大,传统关系型数据库在存储容量、水平扩展、解决分布式问题上都显得无能为力,也就刺激了非关系型数据库的产生。非关系型数据库的存储方式多种多样,以文件存储或者列存储、图存储等方式为主,由于其能够在大规模廉价的集群上存储这一高可扩展特性而深受用户与企业的喜爱^[5]。所以目前大部分企业都会选择适合自身的方式将工作重心由传统关系型数据库转变为搭建互联共享的云平台存储与办公环境,并为类似于 Hadoop 的大数据分析平台提供基础数据支持^[6]。

目前在企业急切想要转型的大环境下，用户不得不面对的是传统业务与关系型数据库耦合性太高的问题，为了将数据平滑地迁移到现有的非关系型数据平台上就需要对新老数据库操作机制以及实现原理、存储方式等有更深刻的学习与研究^[7]。本文从关系型数据库出发进而扩展到非关系型数据库，设计开发出一种便捷、可视化的操作工具，降低数据备份等专业性较高且出错率较高工作的难度，将数据迁移这一繁杂的工作进行简化并适应目前互联网环境日益增多的各种数据迁移需求。

1.2 研究的意义

一个功能强大的数据迁移中间件能够解决目前互联网架构的许多问题：

（1）数据的冷备份

数据可能会因为各种人为或者天灾等因素面临丢失或者被破坏的风险，人为因素例如数据库管理人员的误操作或网络黑客的攻击等，自然因素例如存储设备老化故障或者地震等不可抗力造成存储介质的损坏^[8]。为了避免这种风险，就需要对数据进行冷备份。

伴随着服务器海量数据的不断增长，数据体积变得越来越庞大，这时再进行数据备份与数据恢复速度就会变得尤为重要，假设过亿的数据体量进行备份，会对数据库产生巨大的压力，数据如果不慎丢失恢复起来又需要很长的时间，数据恢复的这段时间用户就不能使用，这就对数据备份与恢复速度提出了更高的要求，传统的数据备份方式就无法胜任现在的需求，这时候高效、安全、高速的数据迁移中间件就成了企业与用户的选择^[9]。

（2）数据异地多活的必要基础

在目前巨大的互联网体量下，为了减轻服务器的压力，可能需要对服务器进行扩容，但是因为没有空余的机架或者其他的原因导致机房已经无法扩容时，则需要新的机房，异地多活具备在生产环境上进行灰度的能力。同时对用户进行分流以缓解服务器的压力也是目前加快服务响应时间的重要方式，许多公司采用了异地多活的数据库设计方案。异地多活指的是在不同地点同时活跃，假设有两个机房，期望北方用户的请求进入北京的机房，南方用户的请求进入武汉的机房，但是问题在于

用户可能今天在北方，明天就在南方了，还有就是用户处于南北区域划分的边界可能会产生漂移，所以南北机房数据的同步就是一个双向复制的过程，为了解决数据循环复制的问题数据迁移中间件也提供了较完善的解决方案。

（3）异步消息通知

目前大部分数据迁移工具仅仅只提供了简单的数据复制的功能，而数据迁移中间件的一大亮点就是同时实现了数据订阅功能，现有的用户需求可能不仅仅是数据的备份而是需要使用提取的数据进行消费，如果同时将复制的数据一起发送给消费端可能会给消费端产生巨大的压力从而导致系统故障等一系列问题，将 kafka 消息中间件进行二次开发并嵌入形成自己的 SDK，用户可以选择使用中间件提供的 SDK 或者 Kafka 等开源消息中间件进行数据的消费，这种将数据生产者与数据消费者解耦的方式也越来越受到用户的欢迎^[10]。

（4）线上 OLAP 系统支持

大数据近年来也成为了人们口中的热门词汇，大数据也可以被理解为大或复杂的数据集，放在目前的市场环境也可以被理解为海量的结构化和非结构化数据，而为了处理这些数据，以存储、运算、展现作为目的的平台应运而生，它们被称之为大数据平台，在大数据平台上进行大数据分析能够显著提升效率、节省时间、减轻压力^[11]。但是面临的问题就是企业用于数据存储的基本上为传统关系型数据库，如何将关系型数据库的数据迁移到大数据平台也就成了用户关心的问题，而这时能够将数据从关系型数据库迁移到大数据平台的中间件也就成为了用户的需求。

（5）分布式数据库数据重分布

在分布式数据库中可能会出现随着数据增长分区键区分度不够高的情况，这时候就需要重新选择分区键并将数据重新分布，由于同时又要维持线上业务正常进行，这时数据迁移中间件就能够提供有效的在线数据迁移解决方案^[12]。

1.3 国内研究现状

数据迁移技术在国内发展较晚，早在 2007 年武剑等人就在介绍数据集成系统的文章中给出了说明并详细介绍了数据迁移中比较重要的 E (Extract)、T (Transform)、

L (Load) 技术。同年于海涛等人也发文详细介绍了基于 Java 技术的异构数据传输过程。在 2008 年廉继红等人将武剑等人提出的 ETL 理论基于 SQL 语句进行实现。随后在 2011 年,丛慧刚等人借鉴前人研究 ETL 的成果并加入自研的元数据处理方式、数据映射模式形成完整的异构数据库迁移工具并在实际使用中取得良好效果。

国内的各大互联网公司都基于各自公司的需求与业务模式研发出了相应的迁移工具,例如腾讯、阿里、百度等。但是对比系统的市场推广,还是阿里研发的 DTS 功能更加全面,市场推广更加成功。例如为了帮助用户构建高可用、易于扩展、更加安全的数据架构, DTS 在大功能上提供了数据迁移与数据订阅功能并基于以上两个功能进行深度开发,目前市场推广版本已经可以实现数据异地备灾、数据不停服迁移、服务异地多活、数据异地同步、实时数据仓库、应用系统缓存更新与消息异步通知等多样化业务场景。

1.4 国外研究现状

由于国外互联网发展较早,所以在数据迁移技术上比较成熟,在 2000 年以 Florescu 为代表的一些研究者就对使用 XML 文档进行数据迁移的方式进行了研究,并指出在迁移效率上相较以前的方法能够取得突破性提升,但是当时并未提供有说服力的实验数据^[13]。在 2002 年 Bouzeghoub 和 Lenzerini 转换了思路,详细介绍了异构数据库间数据如何进行 E (Extract) 与 T (Transform) ^[14]。同年 Tseng 和 Hwung 在 Bouzeghoub 和 Lenzerini 研究基础上提出了数据自动提取并且自动转换格式的技术 ^[15]。2005 年 Williams 等人在完善数据库数据提取方式的前提下对异构数据库数据映射理论进行了论证 ^[16]。2013 年数据迁移系统的设计终于有了完整的结果, RM Marks 针对数据量庞大、数据格式复杂的数据库迁移提出一种更为出色的数据映射方式,而 Wang 等人则深入考虑到数据迁移过程中数据的完整性和一致性并结合现有研究成果设计并实现了异构数据库迁移系统 ^[17,18]。

目前国外市场上比较成熟的数据迁移中间件以及相关产品也比较多, Apache 旗下的 hadoop 就为了方便企业转型提供了 DBInputFormat 方法,使得传统数据库与典型非关系型数据库 hadoop 之间的数据导入导出变得简单,其实现原理也容易理解,

利用主流的 JDBC 将源端关系型数据库的数据进行查询并使用 limit 或者偏移量进行任务划分后转换为 hadoop 能够识别的文件格式并进行传输保存^[19]。

Sqoop 同样是 Apache 旗下的一款产品，主要用作数据迁移，与 hadoop 类似的是为了与传统数据库链接同样在内部嵌入了 JDBC 模块，不同的是在工具使用形式上以命令行为主且增加了 hadoop 不支持的增量数据同步功能^[20]。使用包含全量数据复制以及增量数据同步的 Hadoop 能够使得数据在新老数据库系统中的导入、导出变得简单。

同样，数据迁移的关键技术即数据解析也早就有国外的开源软件产品 tungsten 以及 Kettle 实现，所以目前国外迁移工具的研发进程以及思想已经走在了国内之前，但是相比于国内阿里研发的 DTS，可能在功能实现上没有那么多多样化，这也跟市场需求有很大的关系，随着更多工具的开源化，相信国内外的数据迁移中间件能够各取所长，一起发展。

1.5 本文概括

本文主体内容主要包括以下几个方面，首先简单介绍了涉及的技术，再从功能性与非功能性方面分析了系统需要具有哪些“能力”，进而延伸到中间件的分析与设计，从任务的执行流程顺序入手，逐步分析中间件的设计并抽象出公共模块进行重点讲解，最后对中间件进行了一定的总结并从具体切入点讲解了中间件的一些需要改进的地方。

2 关键技术

2.1 Zookeeper 服务

2.1.1 Zookeeper 服务概述

Zookeeper 是 Apache 旗下的一款软件，主要用于分布式系统，主要功能是提供命名注册、同步服务以及最重要的分布式配置服务，与目前市场上主流的分布式配置服务类似，Zookeeper 也是通过构建冗余为系统提供高可用服务^[21]。Zookeeper 可以应用在如下几个典型的场景中。

（1）数据发布与订阅

数据发布与订阅也称之为配置中心服务，简单说就是用户将信息发布到 Zookeeper 节点上，然后不同用户都可以在这个节点上订阅发布者发布的消息，这样消费者用户能够动态获取发布者用户的有效信息，从而实现配置中心的一种动态数据更新以及用户数据的集中式管理^[22]。

在日常的开发工作中，经常会使用到一些通用的配置信息，这些信息一般具有数据量较小、内容动态变化、在各机器上共享一致等特征，对于这样的数据开发者就可以发布到 Zookeeper 节点上，让共享配置的机器去订阅该节点上的消息，从而减轻开发人员压力并且保证共享配置的正确性^[23]。一般数据发布与订阅有推模式和拉模式两种：

① 推：Zookeeper 节点主动将数据更新发送到所有的订阅者。

② 拉：订阅用户主动请求 Zookeeper 节点下发最新的数据更新，工作节点不会将数据更新推送给未发出请求的订阅者。

Zookeeper 的数据发布/订阅采取的是更为巧妙的推拉结合方式，即工作节点将数据更新的消息推送给订阅者，由订阅者决定是否主动去获取具体变更数据。

（2）命名服务

命名服务也是分布式系统中常见的应用场景，顾名思义就是对资源进行命名以方便用户对资源进行更好定位与管理，其中资源可以是主机名或地址、远程对象或

者应用服务^[24]。以大家熟悉的 JDBC 链接数据库为例，开发者一般不会将数据库用户名与密码等信息直接写在链接语句中，而是将一些固定字符串配置到配置文件中并在代码中通过配置文件读取，例如当下使用火爆的 SpringBoot 框架等直接将数据库链接全部封装。获取配置文件就是一种命名服务即对配置文件进行命名然后通过这个唯一命名获取到这个资源。Zookeeper 命名服务的应用场景多为获取分布式系统中服务地址列表，当然 Zookeeper 能完成这个功能有两个重要的前提即 Zookeeper 节点拥有类似于文件系统的目录结构以及 Zookeeper 能够顺序创建节点^[25]。

（3）分布式协调/通知

Zookeeper 的分布式协调/通知功能其实与消息发布/订阅功能类似，差别在于消息发布/订阅主要用于单个系统而分布式协调/通知主要用于分布式环境下不同机器或者不同系统之间的消息通信，使用方式为分布式环境下不同的机器都在 Zookeeper 某一节点上进行注册，当该节点或者该节点的子节点发生变化就能够及时通知所有订阅者并做出应对策略，这是一种常用的分布式系统通信方式^[26]。

（4）心跳检测

在分布式系统中，心跳检测主要用于各分布式组件之间查看彼此的存活情况，例如在分布式系统中 1 号主机需要知道 2 号主机的存活情况以决定是否将任务信息传递给 2 号主机。在传统的开发方式中，开发者可以采用 PING 的方式或者建立长连接并利用 TCP 固有心跳检测机制来检测不同机器间的连通性。而 Zookeeper 采取的心跳检测方式则巧妙利用了 Zookeeper 临时节点的特性，让分布式系统中所有机器都在某一指定节点下创建临时子节点，这样通过判断这个临时子节点是否存活就可以判断不同机器的存活情况，将所有机器间的耦合性大大降低，同时增强了系统的安全性与可管理性^[27]。

（5）工作进度汇报

在一个分布式系统中，不同机器除了需要知道其它机器是否存活，还需要知道不同机器上的任务执行进度，与 Zookeeper 实现心跳检测的方式类似，当不同机器在 Zookeeper 上某一节点上创建子节点后，实时或者定时将各自的任务执行进度进行上报并记录，这样 Zookeeper 节点就能够实时监控所有组件的任务执行进度并按照需求

将不同任务进度进行通知下发^[28]。

（6）Master 选举

Master 选举也是如今分布式系统中最为重要的功能需求，同时也是最典型的应用场景，开发者常见的 Hadoop 配置中也会配置 NameNode 与 DataNode，当 NameNode 故障后也会进行 NameNode 的选举。

通常情况下进行 Master 选举的方式有多种，一种比较常见的方式是利用数据库主键唯一的特性，所有节点都向同一表中插入一条相同的记录，能够成功插入的节点就成为 Master，但是这种方式的问题在于工作节点无法感知到 Master 节点是否故障。这时候开发者可以利用 Zookeeper 的文件系统特性与节点特性，首先 Zookeeper 会向所有机器发出选举指令，这样所有的客户端机器就会请求创建同一个临时节点，最终只有一个机器能够创建成功，那个节点即为 Master，其他没有创建成功的机器都会在选举中成功创建的节点上注册一个节点变更的 Watcher，当 Master 故障时其他机器能够立即感知到事件并重新进行选举，这样就实现了 Master 动态选举^[29]。

2.1.2 Zookeeper 服务优势

目前由于互联网用户体量过大，系统如果出现故障而停机造成的损失是无法估量的，故而高可用的概念被提出。而 Zookeeper 本身自带 Master 选举功能能够在系统故障的时候使系统能够继续工作，而 ZK 特有的分布式协调与节点工作进度上报功能能够帮助目前的分布式服务实现高可用等重要功能。

2.2 Kafka 消息队列

2.2.1 Kafka 消息队列概述

Kafka 同样是 Apache 公司开发，主要由 Java 和 Scala 编写，是一个用于处理实时数据的开源流处理平台，其特点是高吞吐、低延时^[30]。Kafka 本质上是一个支持发布/订阅的消息队列，只不过在其持久化层适配了分布式事务架构，在“大数据时代”这个特点能够为企业数据流式处理提供非常高的应用价值。此外 Kafka 还提供了多样化的流式处理库用于与外部系统进行数据的输入/输出交流，例如主要用于 Java 系统的 Kafka Stream 库。

经常与 Kafka 消息队列一起被提出的概念有“生产者消费者模式”，生产者消费者模式的含义是生产者生产的数据不直接交由消费者消费，而是存储到一个中间组件中，消费者从这个中间组件中获取数据进行消费，这样一是能够将生产者与消费者解耦，二是能够协调生产者与消费者的速度，当生产者生产速度过快时会阻塞消息生产，当消费者消费过快时会阻塞消费者消费，从而达到处理速度最大化。

而 Kafka 消息队列就是这个中间组件，数据存储到 Kafka 消息队列首先会被分配到不同的“分区”（Partition）、不同的“Topic”下，在一个分区内，这些消息被索引并关联同时戳存储在一起。Kafka 可以运行在集群上，且同一分区可以跨集群节点存储^[31]。

Kafka 不仅仅提供了对外的流式处理库，还可以与市场上使用率很高的 Spark、Strom 和 HBase 集成为集群部署到不同的服务器上，Kafka 有四个主要的 API 来处理发布/订阅消息，分别为生产者 API、消费者 API、Stream API 和 Connector API。由于其出色的流式消息传输能力和容错能力已经成为了目前主流的消息队列^[32]。

Kafka 架构的主要术语包括 Topic、Record 和 Broker。Topic 由 Record 组成，Record 持有不同的信息，而 Broker 则负责复制消息。Kafka 四个主要 API：

生产者 API：支持生产者信息流式发布。

消费者 API：支持消费者信息订阅以及信息流处理。

Stream API：将输入流转换为输出流，并产生结果。

Connector API：执行可重用的生产者和消费者 API，可将 Topic 链接到现有的应用程序。

2.2.2 Kafka 消息队列优势

Kafka 此类消息中间件的本质作用就是消息的异步消费，它不仅仅是能够存储传统意义上的数据信息，在系统中应用到这样一个异步组件能够解决现在很多系统无法解决的问题，例如本文后面会讲解的一推多拉减轻源端数据库压力的设计以及结构迁移同步等问题。

2.3 RPC 通信协议

2.3.1 RPC 通信协议概述

远程过程调用即开发者常说的 RPC (Remote Procedure Call)，实质上是一个计算机通信协议。该协议的主要目的是方便程序员通过一台计算机上的程序通过 RPC 调用另一台计算机上的程序。

在目前各大互联网公司都使用分布式系统的前提下，应用服务的各个组件可能被部署在不同的服务器上，各个组件之间的低延时通信就成为了保证应用服务良好体验的关键，RPC 的实现其实可以基于传统的 HTTP 协议，HTTP 协议由于其良好的跨平台通用特性被广泛应用于目前常见的 B/S 架构产品，但是由于其传输数据报头较长导致性能比较差^[33]。所以随着 RPC 通信协议的成熟，分布式系统的构建就更加容易，极大推动了当前大数据环境下分布式系统的发展并为其扩展性和容错性提供了技术保障。

2.3.2 RPC 通信协议优势

相比于传统的 Http 通信，使用 Rpc 通信的理由也非常简单，在目前的分布式系统架构的基础上，系统之间的通信需要一个更加便捷、更加轻量级、更加快速响应的方式，Rpc 恰恰能够满足这些需求。

2.4 Hadoop 框架

2.4.1 Hadoop 框架概述

Hadoop 同样是 Apache 旗下产品，是一款由 Java 编写的支持数据密集型分布式应用程序的开源框架。它的特点是在分布式集群环境中允许使用简单的编程模型存储并处理大数据，能够在集群的每个机器上都提供存储与计算能力，并且有效地支持服务器的横向扩展，在大数据处理上能极大提升效率并缩减成本^[34]。

Hadoop 的工作原理简单说起来就是一个“分割与合并”的过程即 Hadoop 的 MapReduce 计算过程，它的过程是将任务分割成许多小部分后每个部分都能在集群

的任意节点上执行；同时 Hadoop 还提供了分布式文件系统的功能，通过这个功能能够存储所有节点数据信息并提高整个集群的带宽^[35]。而结合 MapReduce 与分布式文件系统的特性能够使 Hadoop 更加从容应对海量数据存储与计算分析的需求，也成为了目前市场上最火热的开源框架之一。

2.4.2 Hadoop 框架使用优势

关于 Hadoop 平台的优势，可以从以下四个方面来介绍。

（1）高可扩展性

由于 Hadoop 能够有效支持横向扩展，用户或者企业可以将大量廉价服务器组成集群用以处理海量数据下的存储与计算任务而无需为了将服务器设备纵向提升而花费大量资金，在这种集群横向扩展模式下系统的可扩展性有了极大提升^[36]。

（2）高性价比

传统的数据存储模式为存储最优价值数据，因为数据存储介质的纵向扩展会花费大量的财力，而最优价值数据的选择也会耗费企业大量的人力与财力，当公司规模不断变大，在数据存储方面所花费的财力会呈指数型增长，对于企业的发展极为不利，这时 Hadoop 就为企业用户提供了极具成本效益的存储解决方案^[37]。Hadoop 架构被设计为一个横向向外扩展的架构，可以使用廉价的服务器存储所有有价值数据，随着数据量的增大，这种方式能很好地解决问题。

（3）高灵活性

由于 Hadoop 提供的分布式文件系统功能，Hadoop 能够在目前主流的各种社交媒体与电商平台上获取宝贵的信息并加以存储分析进而可以从这些数据中得到商业价值，同时由于其易扩展特性还被广泛应用作各种数据仓库，由于其线上 OLAP 分析系统的支持使之能广泛应用于用户行为分析与欺诈检测等多个领域^[38]。

（4）高处理速度

由于 Hadoop 独特的文件存储方式，使得 Hadoop 能够兼容各种结构化与非结构化的数据，这就天然省去了数据类型转换这一繁杂的步骤，同时由于其 MapReduce 计算方式，海量数据分析计算能够被拆分为多个微小的计算单元单独进行并进行结果整合，在数据分析计算领域较传统系统相比可以说产生了质的飞跃^[39]。

（5）高容错能力

Hadoop 同样使用冗余来保证它的高可用性，但是由于其横向扩展特性，数据冗余的成本也被大大缩减了，当集群中某一节点发生故障时，该节点上的数据都能够在备份节点上被找到^[40]。所以 Hadoop 相比于传统关系型数据库在涉及处理大量数据集的成本与安全性上有更大的优势，其独特的能力与特性可以为企业的发展持续提供价值。

2.5 本章小结

本章主要是对中间件使用到的关键技术以及技术的使用场景、技术的使用优势做了简单的介绍，通过这些内容再结合中间件的需求分析，有助于理解中间件每个模块的设计思路以及辅助技术的选取原因，从而增进对整个中间件的设计架构的认知。

3 中间件分析与设计

3.1 需求分析

从中间件的使用角度，用户分为三类，一类为管理员，一类为私有用户，最后一类为公共用户；管理员用户具有最高管理权限，能够管理所有注册用户，同时能够管理所有注册用户的任务、配置任务失败报警人员信息；私有用户拥有自己的执行服务器，能够查询并管理自己配置的任务，配置自己任务的报警人员信息；公共用户任务在共有服务器上执行，能够查询并管理自己配置的任务，配置自己任务的报警人员信息。

从中间件实现角度，中间件主要需求由功能需求与性能需求组成，本节就从这两个角度进行分析。

3.1.1 功能需求分析

（1）预检查

在任务成功创建后，需要进行一定的检查来确保任务能够顺利执行的，这些检查包括：

- ① 执行节点与源端网络连通性检查。
- ② 如果任务选择结构迁移，则需要检查用户提供的账户是否在目标端有建表等相应的权限。
- ③ 大小写适配检查，即用户设置的映射信息后是否勾选大小写适配，如果用户勾选大小写适配，则类似 Name 与 name 这样的字段分别位于源端和目标端是可以通过检查的，否则预检查就会报错。
- ④ 双向复制检查，如果用户选择开启双向复制，需要检查是否存在标志位以及源端和目标端列的类型。

（2）结构迁移

结构迁移即将源表的结构迁移到目标端，结合中间件的功能使用场景主要分为两种情况：

① 任务开始前结构迁移，此时能够实现结构迁移的方式比较多，例如手动在目标端建表或者使用开源的 DDLScan 等工具自动化建表。

② 任务正在进行时的结构迁移，此时实现的结构迁移比较复杂，需要将源端的结构变化同步到目标端，同时还要考虑到分布式数据库表数据可能存储在不同节点的情况，后续会对具体实现做详细说明。

（3）数据采集

数据要进行迁移，首先需要进行数据采集，才能通过后续设计的各种方式将数据迁移到目标端，中间件数据采集的方式主要分为全量以及增量两种，如下：

① 全量迁移的数据采集方式类似于对源端做一个快照，然后根据主键的不同将任务进行分割并封装，最后交由后续程序执行。

② 根据目前市场上主流的数据库，中间件的增量数据采集方案是根据数据日志进行提取解析，并将提取出的数据进行数据转换封装，并在 `applier` 模块进行应用。

（4）数据过滤

从源端提取的数据有些对于用户而言是无用的，比如用户只想迁移源端的部分字段，或者只想订阅源端的部分字段，这时候需要对现有的数据进行过滤，即需要支持用户自定义设置过滤字段，且这个字段的设置应该是在用户任务启动之前。假设用户在任务过程中修改了过滤字段，由于之前产生的数据可能还未消费或者还未在下游应用，这样不论是订阅任务还是迁移任务都会产生数据不一致情况甚至出现错误，所以数据过滤条件的设置一定是在任务启动之前，且会对数据迁移任务的全量过程、增量过程以及校验过程全部生效，会对数据订阅任务的增量数据生效。

（5）数据应用

将数据进行提取、转换、过滤、封装后就需要对数据进行应用。为了实现异地多活、线上 OLAP 系统支持等越来越复杂的需求，既要保证数据的正确性，同时又要最大化减少数据迁移的延时，就需要更加合理与高效的高并发设计。

（6）数据校验

数据校验是为保证数据完整性进行的一种验证操作，为了确保数据正确进行迁移并更快找出不一致数据与产生的原因，数据校验是中间件必不可少的功能。

3.1.2 性能需求分析

一个完整健全的中间件除了能够实现这些必须的功能外，还需要对性能提出更多的需求以适应市场上的需求，小结如下。

（1）安全性需求

保证用户数据的安全也是中间件的设计重点，目前中间件通过以下几个方面的设计来保证用户数据的安全性：

① 访问控制策略：数据迁移中间件采用租约制度，通俗来说就是分为公有与私有的区别，公有即所有共有用户任务在同一个服务器上或者同一个集群上执行，私有即私有用户的任务是在单独的服务器或者集群上运行的且只运行此用户的任务，每个用户登陆后只能看到自己的任务情况；同时用户可以为自己的服务器设置白名单，只有白名单的 IP 地址所属设备才能访问。

② 建议用户通过内网访问服务器部署实例，可以使服务器免受 DDoS 攻击的风险。

③ 若用户选择对外开放，当中间件认为用户实例正在遭受 DDoS 攻击时，会首先启动流量清洗功能，即当监测的 PPS（Package Per Second）或者 BPS（Bits Per Second）过高或者每秒新建并发连接、非并发连接过高就会进行流量清洗，处于流量清洗状态下的应用实例可以正常访问。

（2）可靠性需求

数据的可靠性是数据库迁移中间件的必要条件，如果数据在迁移过程中产生了丢失或者损坏，那对于企业来说会造成灾难性的后果，为此中间件也进行了多种设计来保证数据可靠性：

① 进行全量或者增量迁移的任务会实时上报迁移进度并落库，当服务器宕机时能够找到位置点进行恢复。

② 迁移任务实现采取幂等操作，在任务恢复时能保证数据结果的一致性。

③ 任务结束时有可选择的校验选项来保证任务结果的正确性。

（3）高可用需求

为了使中间件拥有更高的可用性，中间件从服务器高可用以及任务高可用两方

面做了设计。在服务器高可用方面，当中间件的 Engine 节点（即任务执行服务器）发生故障时，能够通过 Center 中心（即 Zookeeper 调度节点）进行任务的 takeover，这样能够保证任务的正常进行；在任务方面，中间件设计了任务断点续传、任务暂停、任务配置修改等多种功能用以满足用户在不重建任务下的各种需求来实现迁移任务的高可用。

（4）高速度需求

仅仅是数据备份容灾对于中间件的迁移速度并不会提出很高要求，但是对于数据异地多活、实时数据仓库、线上 OLAP 分析系统来说，最大化降低数据传输延时是中间件需求的重中之重，同时为了保证数据正确性，从数据的并发复制上去寻求更加安全、可靠、高效的并发策略，才是最优的解决方案，这样在大量数据需要复制的时候，中间件并发处理才能在短时间内解决。

3.2 总体设计与任务属性

3.2.1 中间件总体设计图

中间件总体设计如图 3-1 所示，中间件存在两种接入方式，一是可以通过中间件提供的可视化操作界面，调用相应的 API 进行操作，或者将中间件内嵌到用户的平台上（类似大数据操作平台）。如果用户使用中间件提供的操作界面，那么通过中间件的 API 接口可以连接到配置好的调度中心，调度中心获取用户输入的数据后通过特定任务调度算法将任务下发到执行节点，执行节点根据用户输入信息去执行数据迁移/订阅任务；当然用户也可以选择将中间件内嵌到用户的系统中，通过用户的系统与调度控制中心进行通信并下发任务信息。图 3-1 中与 Center 进行通信的即为接入平台，Center 将任务下发给迁移/订阅执行节点组执行。由于 Center 中心主要由 Zookeeper 实现，自身具有高可用特性，而结合使用 ZK 的心跳检测、分布式协调与任务进度汇报功能能够实现 Engine 任务执行节点的高可用以及任务的高可用，这就是完整的中间件形态。

本节所包含的内容为 Center 与 Engine 的接口定义，任务属性以及 RPC 通信的数据结构，包括创建任务、暂停任务、恢复任务、结束任务、修改任务、信息上报等。

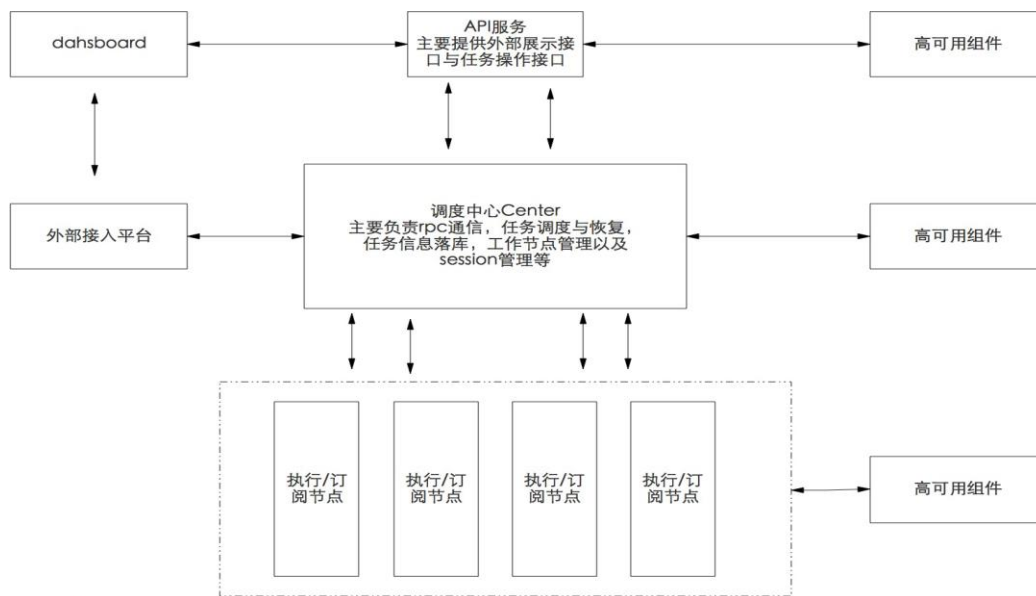


图 3-1 中间件总体结构图

3.2.2 任务模型

Engine 的任务分为三类：

1) 租约机制相关，握手、注册、获取相关配置、续租

(1) 表 3-1 为握手 handShaker 的基本属性与接口

表 3-1 心跳机制属性与接口

参数	类型	说明
nodeType	NodeType: ENGINE, CENTER	请求参数，表示节点的类型
centerHost	String	请求参数，表示 center 的 ip
localIp	String	返回参数，表示节点 ip
minPort	int	返回参数，表示最小的 port
maxPort	int	返回参数，表示最大的 port

(2) 表 3-2 为注册的基本属性与接口

表 3-2 注册机制属性与接口

参数	类型	说明
Session	Session 包含的信息	请求参数，Session
nodeId	int	返回参数，节点 Id
sessionId	long	返回参数，SessionId

(3) 表 3-3 为获取全局配置的属性与接口

表 3-3 获取配置属性与接口

参数	类型	说明
sessionId	long	请求参数, Session 的 ID
touchTimeout	long	返回参数, 上报状态的超时时间
touchInterval	long	返回参数, 上报状态的周期
rpcTimeOut	long	返回参数, rpc 的超时时间

(4) 表 3-4 为续租和上报状态属性与接口

表 3-4 续租上报状态属性与接口

参数	类型	说明
sessionId	long	请求参数, Session 的 ID
Statistic	ReportEngineStat	请求参数, 返回 Engine 的状态以及所管理 job 的状态

2) 操作相关, 增加任务、修改任务、开始任务、暂停任务、取消任务, 表 3-5 为操作属性与接口。

表 3-5 操作属性与接口

参数	类型	说明
DtsJob	DtsJob	请求参数, 一个 dts 任务的所有消息, 可用来修改任务
JobRuntime	JobRunTime	任务的运行信息
JobId	Long	请求参数, 可用来启动任务, 暂停任务, 恢复任务

3) 监控信息推送相关, 推送 engine 中的 job 信息, 表 3-6 为 Engine 租约机制相关对象。

表 3-6 租约相关属性

类	属性	接口
GlobalConfig	TouchTimeout, touchInterval, rpcTimeOut	无
EngineConfig	MaxJobSize, maxcJobDisk, rpcProperties	无

一个 Job 的状态有 8 种，如下表 3-7 所示，任务处于不同的状态代表了不同的含义，同样各任务状态之间也不是随意转换的，有些状态变化是用户操作导致的，而另一些则是任务运行过程中的自然转变，通过这种任务状态设定能够使开发管理人员更快查找任务处于哪一运行阶段并定位问题点。

表 3-7 Job 状态表

状态	描述	是否占用 engine 资源	任务后台执行状态描述
Init	初始化	否	任务刚创建
Ready	启动中	否	任务已经下发到某 Engine 上，并未真正启动。 该状态是一个中间状态，正常情况后续一定会被更新成 Failed 或者 Alive
Failed	失败	不一定 如若为下发失败，则不占 engine 资源 若为执行失败，则占用资源	失败状态下，需要标注任务失败原因，如大体可分为下发失败、启动失败、执行失败 失败状态下，任务依旧占用 engine 资源 该状态下，用户可通过操作界面点击【恢复】启动执行
Alive	运行中	是	正常运行
Suspend	挂起	是	任务被用户挂起，该状态只会由用户点击【暂停】触发 挂起状态下，任务依旧占据 engine 资源
Complete	完成	否	任务完成，目前只存在于全量任务
Stopping	停止中	不一定	center 已经向 engine 发起停止任务的请求，但是未收到任务确实上报 stopped 状态的回馈，处于等待
Stopped	停止	否	停止后，任务不再占据 engine 资源。 任务被用户停止，该状态会由用户点击【停止】触发。 高可用逻辑中，center 在进行任务 take over 完成后，会触发原 engine 上目标任务的 stop，但该 stop 操作对用户来说不可见。 停止状态下，用户可以点击【启动】重新下发，下发后的任务按照当前执行位置点继续执行。

各种状态之间的转换如图 3-2 所示。

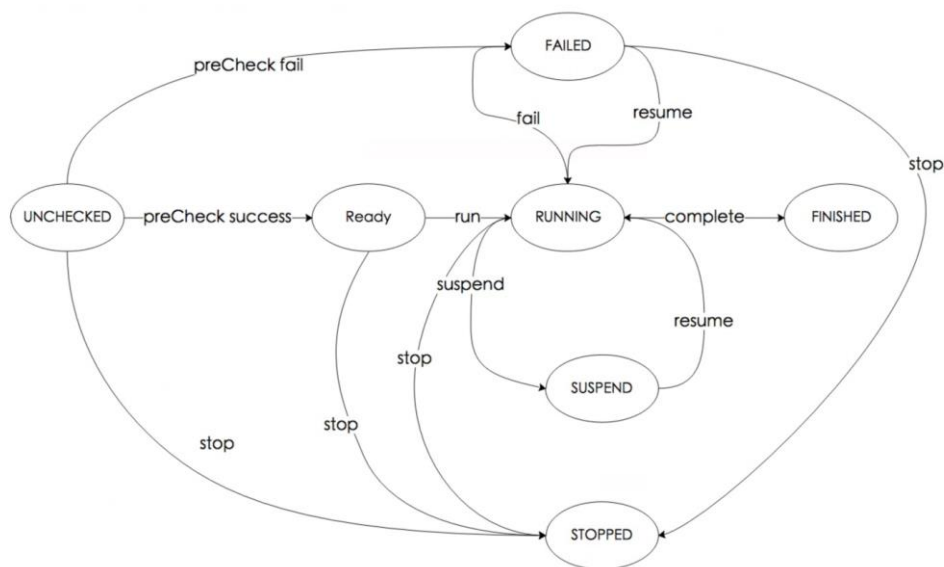


图 3-2 Job 状态转换图

3.3 Center 控制中心设计

3.3.1 Center 总体设计

Center 是 dts 集群的管理中心，功能包括节点管理（节点在内存以及数据库中的状态维护）、租约管理（节点的注册、过期、续租以及状态上报）、任务管理（任务的状态维护、任务新建、调度、启动、停止等操作）。根据任务经过 Center 处理的过程做出如下流程图。根据任务的处理流程，任务通过外部接口或者 dashboard 调用 rpc 将信息传递给 Center，所以 Center 必须包含 rpc 通信模块；任务信息下发之后需要经由 Center 的逻辑进行分发，同时在任务失败后需要对任务进行恢复或者在 Engine 工作节点失联后对任务进行 take over，所以 Center 必须包含一个 JobController 模块和一个 Engine 节点管理模块；任务信息经由 Engine 上报后需要落库，所以 Center 需要包含一个用于数据落库的模块，称之为 dao 层模块；这些必要功能结合 session 管理等小功能一起组成了中间件的 Center 调度中心，任务经过 Center 的流程如图 3-3 所示。

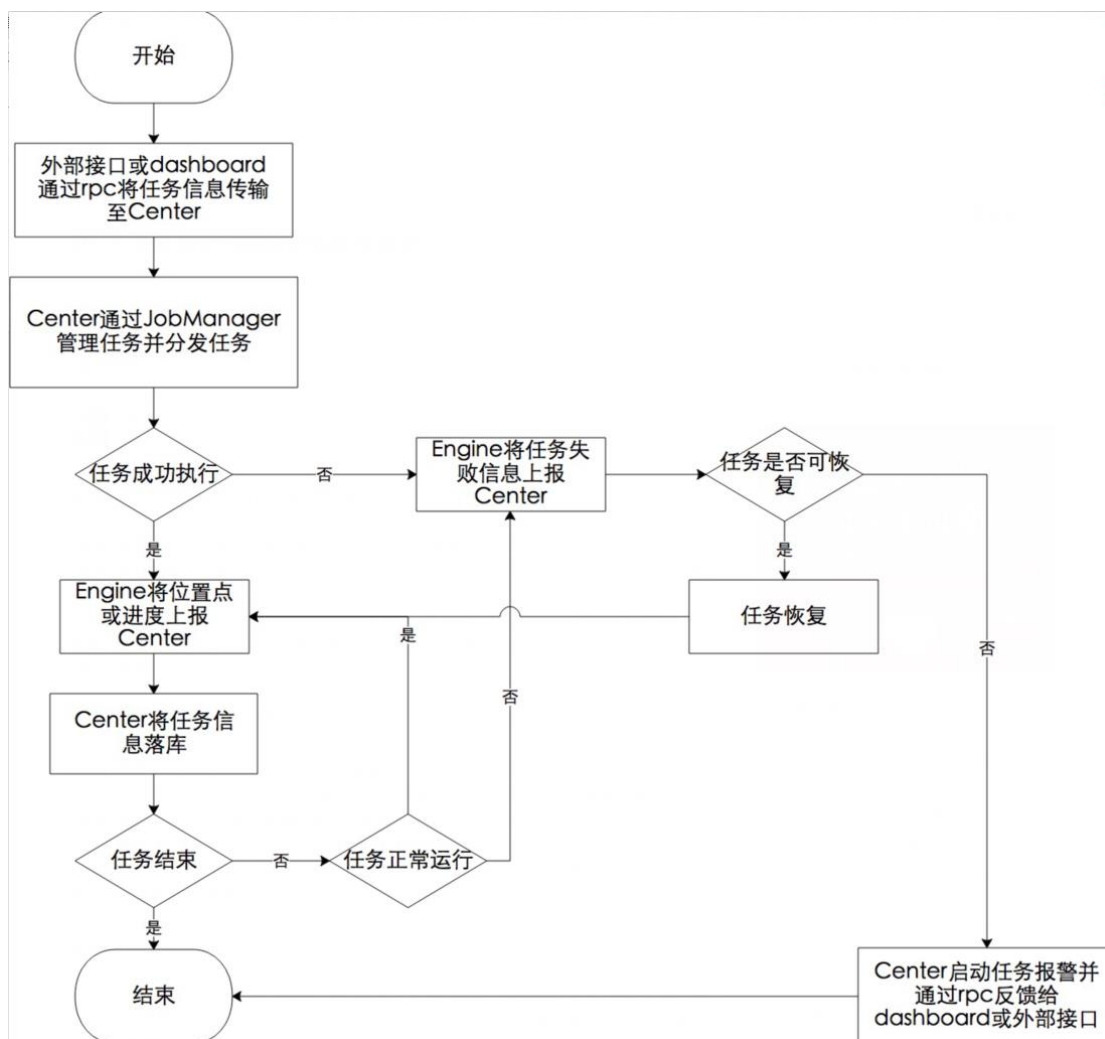


图 3-3 Center 流程图

3.3.2 高可用设计

高可用 HA (High Availability) 是分布式系统架构中必须考虑的因素之一，假设用户在 100 个小时的有效时间内能够正常使用系统的时间为 90 个小时，那么系统的可用性就是 90%，可用性会对用户的使用体验产生巨大影响，所以目前很多公司将高可用的目标定为了 4 个 9 即保证系统全年中在 99.99% 的时间内是可用的，换种说法就是系统全年故障总时间不能超过 8.76 个小时，系统的高可用设计能够极大提升用户的使用体验。

高可用设计在整个中间件中分为 Center 高可用、Engine 高可用以及任务高可用。Center 的高可用由 Zookeeper 自己维护；Engine 高可用指的是 engine 宕机、重启、

失联时，任务能自动 take over，下发到其他 engine 上去执行；任务高可用指的是任务失败后，检测任务 Engine 是否可使用，如果可用则将任务进行断点恢复；若 Engine 不可用或者 Engine 与 Center 失联，则将 Engine 上的任务进行 take over，根据特定均衡算法查找到适合的工作节点进行任务恢复。

中间件将任务高可用过程分为两个逻辑，第一是判断任务是否可以进入高可用队列，如图 3-4 所示。

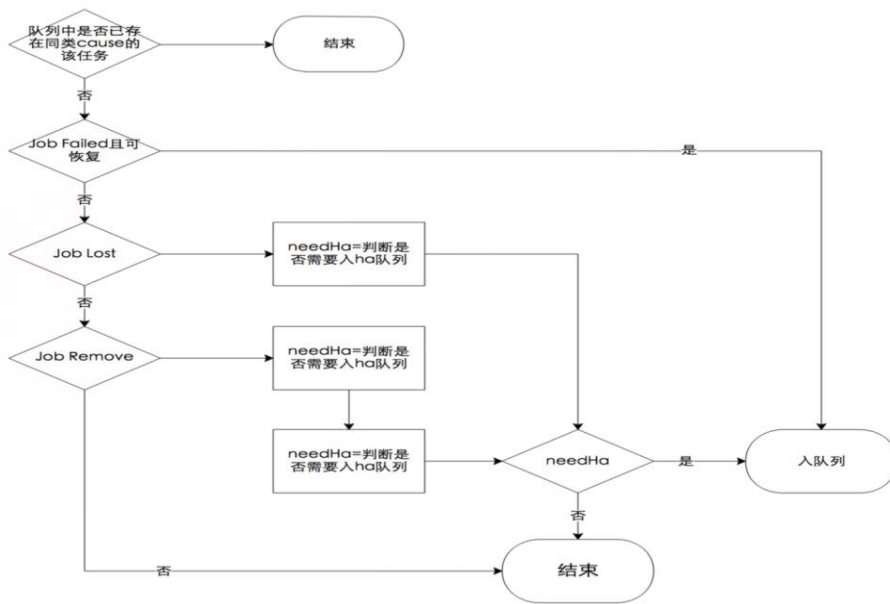


图 3-4 任务入高可用队列流程图

任务具体是否可以入高可用队列的逻辑如图 3-5 所示。

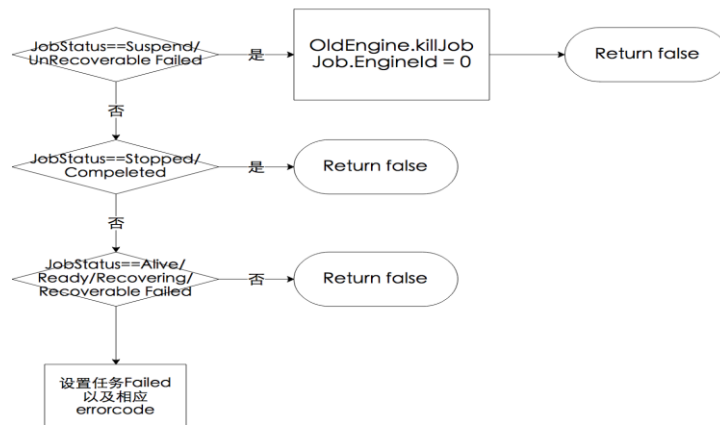


图 3-5 入队列判断流程图

第二则是队列中具体已有任务的执行过程，如图 3-6 所示。

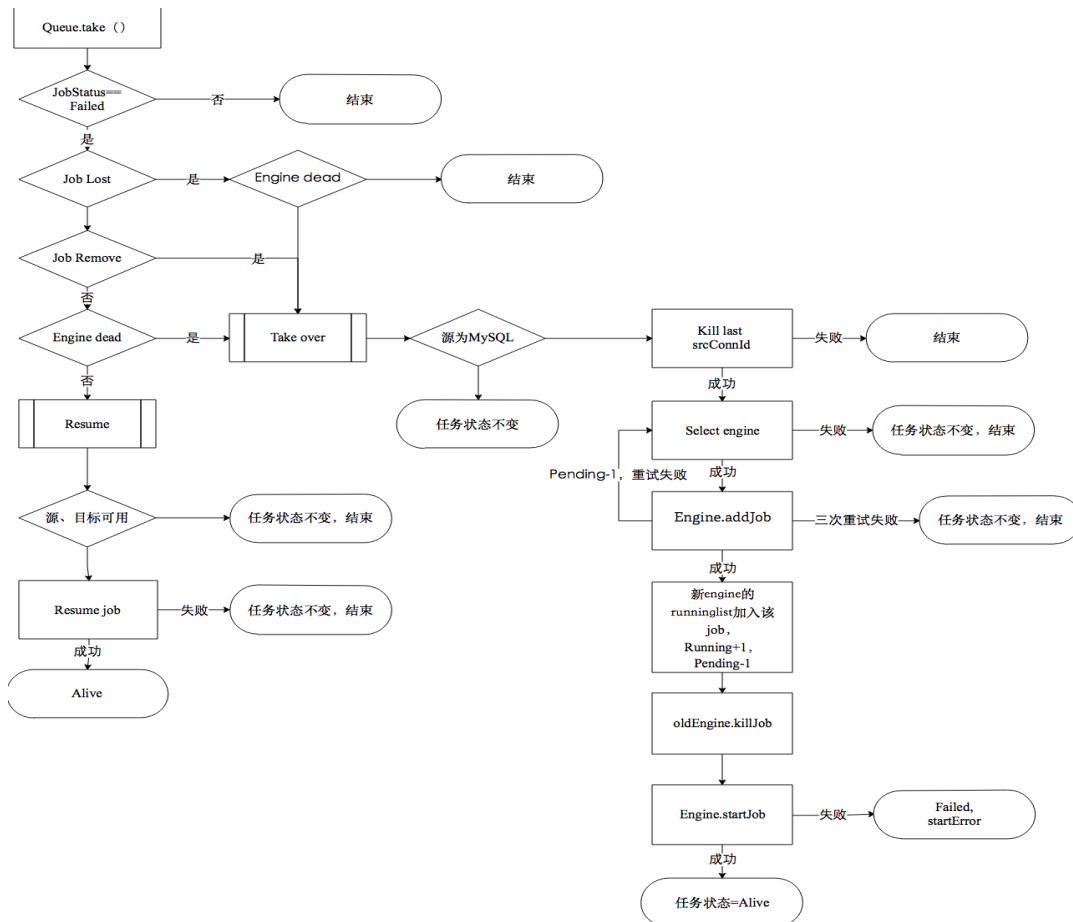


图 3-6 队列任务处理流程图

3.4 Engine 工作节点设计

3.4.1 Engine 专有名词解释

Engine 专有名词解释如表 3-8 所示。

表 3-8 Engine 专有名词表

Engine	中间件中的一个工作节点, 对应一个 JVM, 一个 Engine 上可以运行多个 Job
Job	中间件上的一个任务, 一个 Job 可以有多个 Process, 多个 process 是线性执行顺序, 必须等待前一个 Process 执行完毕后, 后一个 Process 才可以开始执行 (例如全量复制完成后才可以进行增量复制)
JobProperties	Job 的一些属性, 如 Job 中包含那些 Stage

续表 3-8 Engine 专有名词表

JobStatus	Job 的一些状态，体现出 Job 的执行情况，例如正在执行的 Stage 的编号
Process	中间件中的一个子任务，例如某个 Job 中的结构迁移、全量迁移和增量迁移就是三个独立的 Process
ProcessProperties	Process 的一些固有属性
ProcessStatus	Process 的状态信息
Applier	数据复制过程中应用到目标端的过程
Event	数据复制过程中源端的一些操作，在复制过程中经过 Applier 应用到目的端

3.4.2 Engine 总体设计

Engine 主要负责任务的执行，所以任务流程中包含的任务预检查、数据采集、数据转换、数据应用、数据校验等过程都是 Engine 的主要功能，由于这些模块都是系统的设计重点，所以将这些模块分别提取出来单独讲解，本章中主要针对 Engine 的上层设计即全量迁移、增量迁移、数据订阅以及 Engine 的设计优化即一拉多推方案进行讲解。

Engine 的整个工作流程如图 3-7 所示，Center 将任务通过 Rpc 进行下发，Engine 中的 JobManager 模块接受到任务信息后进行任务封装，Engine 将任务通过 ETL 流程进行执行。

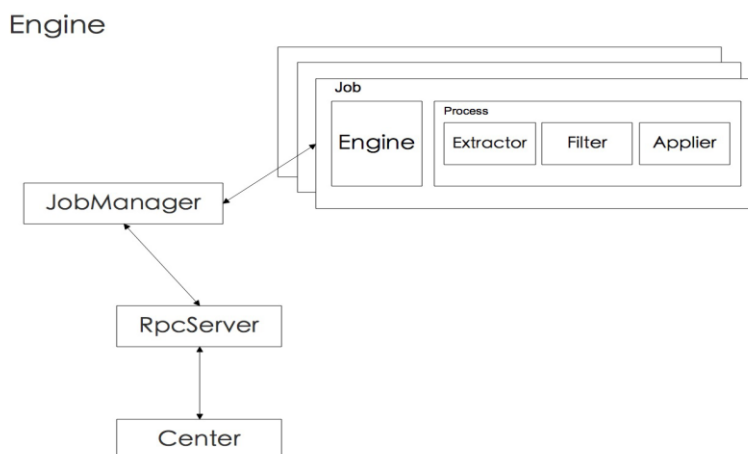


图 3-7 Engine 架构设计图

3.4.3 全量迁移设计

图 3-8 为设计逻辑图。

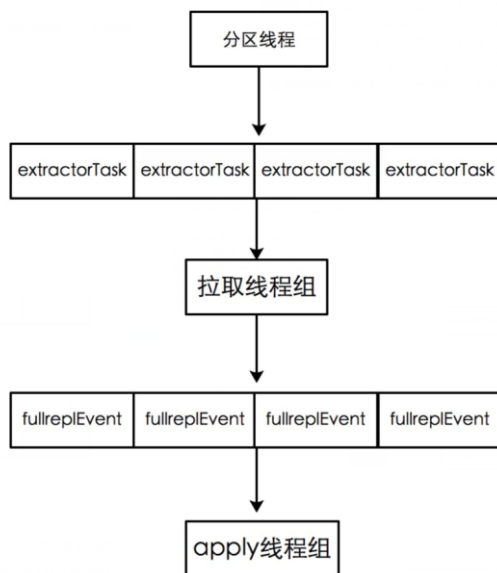


图 3-8 全量迁移线程模型图

上图中，分区线程即为主线程，生成 ExtractorTask，放入 extractorTaskQueue 中，extractorTaskQueue 为有界阻塞队列，队列大小为拉取线程个数；拉取线程从 extractorTaskQueue 中按照 extractorTask 的描述从表中拉取数据，生成 FullReplEvent，放入 fullEventQueue，fullEventQueue 作为阻塞队列，队列大小为拉取线程个数；apply 线程从 fullEventQueue 中拿 fullReplEvent，解析拼接 sql 并 apply 到目标端。

分区对目标的 pipeline 进行循环判断，形成分区生成 ExtractorTask，投入 task queue 中。ExtractorTask 的类结构大致如表 3-9 所示。

表 3-9 task 结构表

参数	含义
schemaName	库名
tableName	表名
startPos	开始迁移位置点
endPos	结束迁移位置点

ExtractTask 的生成逻辑为:

1) 表的主键为 int 类型, 则将源端数据进行 chunk 分区, 每个 chunk 对应一个 task, 每个 chunk 的数据量按照主键差值至少跨 100,000。单表的分区数最大为 1024。即 int 型主键分区的逻辑为:

(1) 查询源表主键最大值 maxPkValue, 主键最小值 minPkValue, $totalSize = maxPkValue - minPkValue$, 默认分区 $chunk = 1000000$;

(2) 计算分区数 $taskCount = \text{Math.ceil}(totalSize / chunkSize)$, 若 $taskCount > 1024$, 则调整 $chunkSize = \text{Math.ceil}(totalSize / 1024)$, $taskcount = 1024$;

(3) 以 taskCount 和 chunkSize 对源表进行切分, 则切分为类似 $[minPkValue, minPkValue + chunkSize - 1]$, $[minPkValue + chunkSize, minPkValue + 2 * chunkSize - 1]$ $[minPkValue + n * chunkSize, maxPkValue]$ 。

2) 表的主键不为 int 类型, 查询最小最大值为 minPkValue, maxPkValue, 生成的 ExtractorTask 起始点为 $[minPkValue, maxPkValue]$ 。

拉取线程组并发从 extract task Queue 中取出 task, 并拼装 batch select sql, select 出的数据封装成 fullReplEvent 放入 fullEventQueue 中, 让下游消费。拉取线程会保证在下一个 task 之前, 上一个 chunk 拉取出的数据在被 apply 线程处理。Apply 线程组从 fullEventQueue 中拿 fullReplCount, 并拼装 insert sql, 插入到目标端, 封装后的 Event 属性如表 3-10 所示。

表 3-10 Event 结构表

参数	类型	含义
schemaName	String	库名
tableName	String	表名
columnValueList	List<ColumnValue>	封装后 Event 包含的行信息
minPkValue	String	最小主键值
maxPkValue	String	最大主键值

任务在全量阶段失败后，需要从失败前的状态恢中恢复，即前提需要任务运行中记录全量迁移位置点信息即恢复点信息（currentPk， minPk， maxPk， activeTasks， waitingPk）：

（1）如果任务源表主键类型非 int，则该表只有一个 task，记录该 task 的推进点即为 currnetPk。

（2）如果任务源表主键类型为 int，则该表可能被分为多个 task，且每个 task 之间主键差距很大，需要分别记录当前活跃的 task 的推进点，即每个 task 对应这 一个 currentPk。

由于拉取线程能保证 extract 下一个 task 前，上一个 task 的数据已经在处理或者处理完成，则当前活跃的 task = [extract 线程组正在拉取的 task]U[apply 线程组正在执行的 task]。假定有个表 t1，（简单起见，假定 chunkSise = 5），minPk=1，maxPk=30，则按照分区逻辑被分为[1-5]、[6-10]、[10-15]、[15-20]、[20-25]、[26-30]，拉取并发数=2，迁移并发数=2，在某个时刻，各个 task 的执行状态可能如图 3-9 所示。



图 3-9 task 执行状态图

则表 t1 当前活跃的 task=[6-10]、[11-15]、[21-25]，对应的 task 恢复点 currentPk 可能分别为（8， 11， 23）。表 t1 未被消费的 task 起点为 26，则表 t1 对应的 pipeline 信息如表 3-11 所示。

表 3-11 pipeline 信息表

currentPk	8（取最小的活跃 task 恢复点，前端显示用）
waitingChunkPk	26
waitingChunkId	6
minPk	1
maxPk	30
chunkSize	5
activeChunks	[6, 8, 10], [11, 15, 11], [21, 25, 23]。每项由该 chunk 的[开始点、结束点、恢复点]的三元组构成

落库具体内容如下：watingPk, minPk, maxPk, chunkSize 均写在 pipeline 表中，activeChunks 单独记在 chunk 表中，chunk 表的字段如{jobId, pipelineId, chunkId, startPk, endPk, currentPk}，联合主键{jobId, pipelineId, chunkId}。

任务如果在上述运行时失败，则从数据库中恢复点的恢复逻辑为：

- （1）将 activeChunks 对组成对应的 chunk，放入 chunk queue 中，即[8-10]、[11-15]、[23-25]。
- （2）从 waitingChunkPk - maxPk，重新按照原分区规则恢复出未被消费的 task，即[26-30]。
- （3）即从 taskQueue 中恢复的 task 如图 3-10 所示。

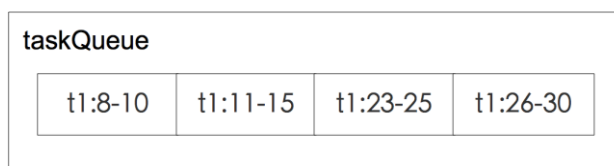


图 3-10 task 恢复图

Pipeline runtime 统计

全量阶段主要需要统计 pipeline 的已迁移数目、currentPk 值，其中比较复杂的为 currentPk 值以及迁移进度，记录 currentPk 的思路如下。

每个 extractTask 对应着 ChunkCheckPointMap，即每个 task 的 chunk 对应的消息积压情况，ChunkCheckPointMap 主要结构如下表 3-12 所示。

表 3-12 ChunkCheckPointMap 结构表

参数	类型	含义
fullTableName	String	表的全名称
checkPointMap	CheckPointMap	检查点结构
chunkId	int	表分区 ID

拉取线程每拉取一个 fullReplEvent, 则往对应的 chunkPointMap 中加入该批次的最小 minId, apply 线程每消费完一个 fullReplEvent, 则从对应的 chunkPointMap 中移除此次消费批次的最小 minId, 则 chunkTask.currentPk = checkPointMap 中的最小值。

pipeline 迁移进度: pipeline 迁移开始时, 从源端获取 totalTableSize (show table status), progress = 0; 在 pipeline 迁移过程中, apply 每消费完一个 fullReplEvent, 则 fullMigSize+=fullReplEvent.Size, 此时 progress = fullMigSize / totalTableSize * 100; 需要注意的是, 由于 totalTableSize 为不精确的值, 所以迁移过程中可能发生 fullMigSize > totalTableSize, 此时 progress 会被强制写为 99%, 所以需要程序中判断 pipeline 迁移完成后, 将 progress 设置为 100%, pipeline 迁移完成的标识为: 该表对应的 chunk 都被 extract 完成且每个 chunk 产生的 fullReplEvent 都被消费完。

全量并发度设置有两个值:

- (1) 拉取并发度
- (2) 迁移并发度

运行过程中均可以修改这两值, 修改后, 任务的相关线程组数据更改, chunk 分区策略不变, 因此数据库中还需要保存 chunk 分区策略相关字段。

3.4.4 增量迁移设计

增量迁移与全量迁移类似, 只是在位置点记录设计上稍有不同, 下文将主要讲解双向复制的设计。

设计如图 3-11 所示。

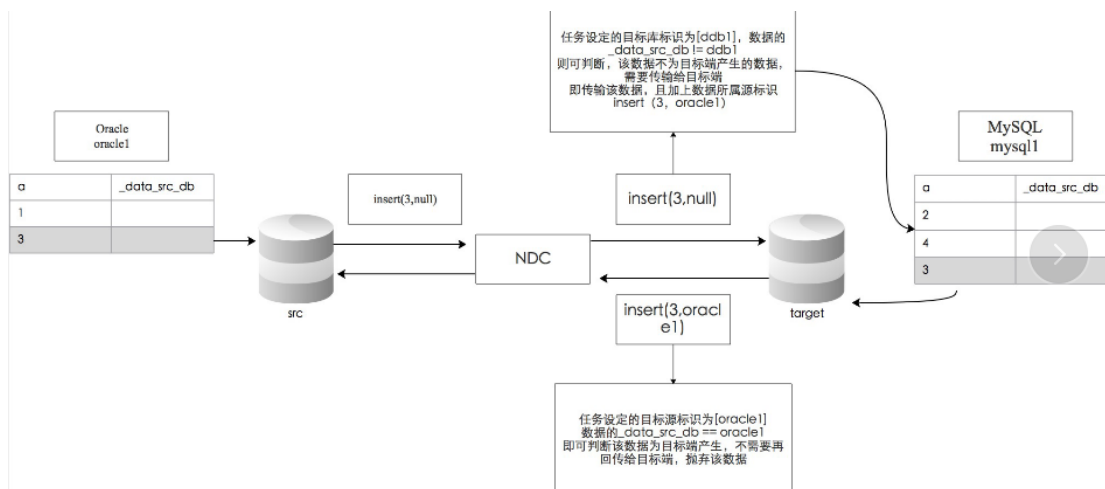


图 3-11 双向复制设计图

需要双向复制的表，需要额外增加一个保存字段 `_data_src_db varchar(255)`，同时由于双向复制的设计会对之前的中间件设计造成影响，例如在预检查过程如果任务选择了双向复制则预检查会检查表上 `_data_src_db varchar` 字段是否存在，若不符合则预检查报错；数据校验也需要忽略掉保存字段的对比。

3.4.5 数据订阅设计

1) 订阅消息序列化方案

在中间件的数据订阅功能中，源端的数据变化被捕捉到之后将推送入消息中间件中，SDK 端再从消息中间件中消费这些包含数据变化的消息。原本为 java 对象的消息需要通过网络进入消息中间件，再从消息中间件传输入 SDK 客户端中，中间将不可避免地经历序列化与反序列化过程。序列化端暂时定为 java，反序列化端使用的语言由 SDK 支持的语言所定。所以序列化方案必须满足跨语言的要求。另外快速的序列化算法和尽量小的序列化后体积也是至关重要的。综合上述问题，一套自定义的序列化方法才能比较好的满足需求，所以我们序列化的规则如下：

- (1) 序列化后为一个字节数组
- (2) 超过一个字节的的数据按照大端排序
- (3) 对象按照其属性顺序挨个进行序列化（不保存属性名称等元信息，所以要求属性的序列化顺序严格与申明时一致）

(4) 数据被划分为两大类：字节数组和字符串，不利于直接展示的数据全部以字节数组的方式存储，可以直接展示并易于阅读的数据以字符串的形式存储

(5) 字节数组和字符串数据都由长度数据（字符串数据的长度其实为编码后产生的字节数组的长度）和内容数据两部分组成

(6) 长度数据占用的长度可能为 1 字节、3 字节、4 字节、9 字节，具体规则为：小于 251 的由 1 个字节表示；251 表示 null 数据；大于 251 小于 65536 的长度第一个字节存储 252，后 2 个字节存储长度；大于 65536 小于 16777216 的长度第一个字节存储 253，后 3 个字节存储长度；大于 16777216 的长度第一个字节存储 254，后 8 个字节存储长度

(7) 字节数组的内容数据由字节数组组成

(8) 字符串数组的内容数据由字符串通过预先设定的编码方式转换为字节数组后组成

(9) 集合（数组、List、Set）先序列化 4 字节长度，然后挨个序列化集合中的元素、顺序与迭代器取出的顺序保持一致

(10) 最开始包含 4 字节的头部信息：1 字节版本号、1 字节校验和信息和 2 字节的保留字

(11) 最后面包含 8 字节的校验和信息（可选）

(12) 枚举类型按照其数量序列化为 1 字节、2 字节或者 4 字节

数据订阅使用格式如表 3-13-表 3-15 所示。

3 个表中的属性设计都是为了保证数据正确性，具体含义也在表中进行了讲解且表结构简单本文就不逐一讲解。

表 3-13 SubscribeEvent 结构表

参数	类型	含义
seqno	long	用于数据去重
lastEvent	boolean	判断 event 是否为上一个
timestamp	long	时间戳
rowChanges	List<OneRowChange>	变化数据封装成行

表 3-14 OneRowChange 结构表

参数	类型	含义
schemaName	String	库名
tableName	String	表名
type	RowChangeType	封装行类型
columnChanges	List<OneColumnChange>	变化数据封装成列

表 3-15 OneColumnChange 结构表

参数	类型	含义
columnName	String	列名
type	ColumnType	列类型
oldValue	object	旧值
newValue	object	新值

2) SDK 并行消费设计

SDK 在之前的设计中，为单线程版本，从 kafka 获取消息，再到调用用户接口处理消息，均为串行处理。为了提高消费消息的速度，在消息处理时采用多线程的方式，在不保证消息处理顺序与接收顺序一致的情况下，并行处理消息。设计如图 3-12 所示。

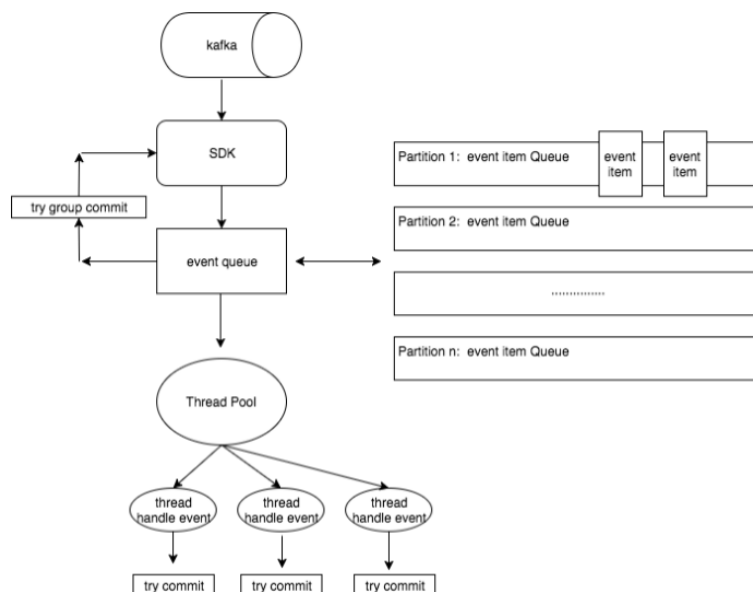


图 3-12 订阅消息并行处理图

(1) SDK 从 Kafka 中获取 event, 并将 event 投入 EventQueue 中, 由线程池异步消费, 处理 event 的并行度为 SubscribeClient 中设置的 concurrency。

(2) 消费消息时, 会记录某个 TopicPartition 中所有为提交的 event (eventItemQueue:按处理的先后顺序排队) 和该 Partition 上次的提交时间。

(3) 当消息处理完时, 从该消息所属 TopicPartition 中的 eventItemQueue 中删除该消息, 如果该消息小于 eventItemQueue 中的对头消息的 offset (说明前面的消息已经提交), 并且该 TopicPartition 上次提交的时间为小于当前时间减去 2s, 则该消息可以进行 commit, 否则不进行 commit。

(4) 当 eventItemQueue 中的消息数大于等于 SubscribeClient 中设置的 maxUncommitCount 时, 该 TopicPartition 中消息的消息会被阻塞, 直到为消费消息数小于 maxUncommitCount。

(5) 当所有 eventItemQueue 中没有未提交消息时, 会进行 group commit

3) 组订阅设计

设计预期是为了达到在任务创建后, subscribeClient 创建时设定其 group, 按照指定 group 对任务进行订阅。

则可执行不同 group 的 subscribeClient 订阅同一任务, 组订阅下, subscribeClient 可设定其 group, 如 g1, 则该 clientRuntime 以及 partitionRuntime 里的 group 均为 g1。

subscribeClient 的启动流程如:

(1) 新建 subscribeClient, 设定 group

(2) 设定任务 url, 向 center 注册

(3) 注册成功, 则开始订阅, 注册失败 (center 会校验 url 以及 group 是否合法), 则抛出错误

3.4.6 一推多拉设计

目前中间件的任务执行架构为, 一个任务对应一个任务进程, 负责从源端拉取数据并投入目标端 (数据存储系统、Kafka), 如图 3-13 所示。

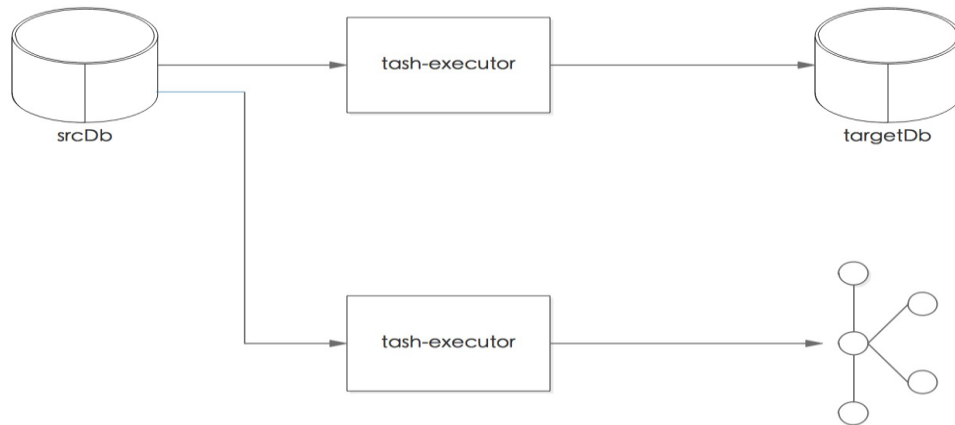


图 3-13 任务执行架构图

该设计有以下两个明显缺陷：

(1) 若对同一源有多份订阅、迁移、同步的需求，则用户在该源上建了 N 个任务，则中间件会在该源上建 N 份复制链接，增加了源端的压力且源的复制链接数目不可控。

(2) 若一个同步任务中同步了多张表，则若其中一张表迁移出现了问题，则整个同步任务处于失败中，导致两个风险：源端 binlog 被回卷；其它表的迁移进度停止。

故而设计以下替代方案，如图 3-14 所示。

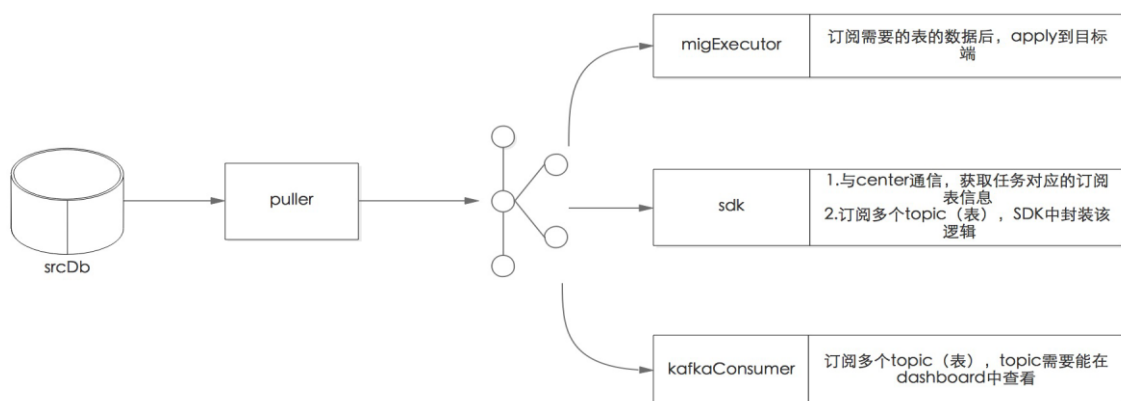


图 3-14 一推多拉设计图

这样的设计方案能够减少网络消耗导致的延时，多表之间互相不干扰，且将数据的中间“存储”交由消息中间件完成减轻服务器压力；但是弊端也比较明显，首先就是强制使用 kafka，若任务量过大，可能导致 kafka 里 topic 数量过多不利于 DBA 维护；同时由于不能保证表的顺序无法满足顺序订阅任务的需求；最后就是同一源的任务必须投递到同一 kafka，否则就需要 puller 可以同时投递多个 kafka。

3.5 Applier 模块设计

3.5.1 Applier 模块设计概述

Applier 模块本身作为 engine 的一个子模块，包含很多功能，主要就是实现数据的高并发应用，但是增量数据在并发应用的时候需要考虑到 Event 的依赖性即可能 event1 与 event2 同时对某一字段进行了修改但是 event1 的修改在 event2 之前，在 event 并发执行的时候就必须严格遵守这个顺序，所以中间件设计了 EventGraphManager 这个结构将包含事件依赖的 Event 封装成链式结构以达到更高地数据迁移速度。

3.5.2 Applier 模块流程图

Applier 模块流程图如图 3-15 所示，所有执行时间都需要经过中间件设计的 EventGraph 结构以判断事件是否独立可执行，图示 3-15 中同样展示了工作线程模型，该线程模型是实现中间件高并发的关键。

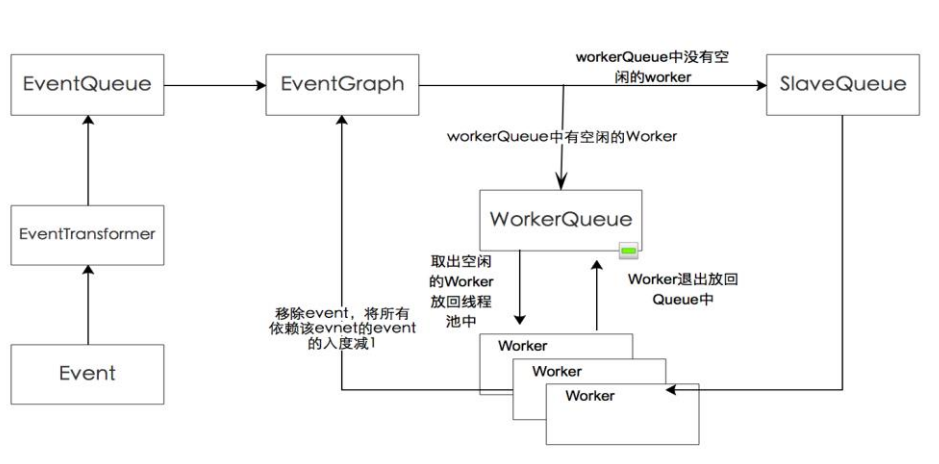


图 3-16 Applier 流程图

3.5.3 Applier 模块功能

(1) EventTransformer 是事件转换器,用于从 Tungsten Replicator 中抽取的数据格式与 Engine 设计的数据格式之间的转换。

(2) EventGraph 是事件依赖图,构建事件彼此间的依赖关系,每张迁移表的每个唯一性索引对应一张依赖图,并由此决定事件是否可以并行执行。

(3) SlaveQueue 是内存中的阻塞队列,只有彼此不依赖的事件才可能同时存在于该队列中,EventGraph 是队列的生产者,Worker 是队列的消费者。

(4) WorkerQueue 是内存中的队列,维护了可用的 Worker,当依赖图有事件移除,首先查看 WorkerQueue 中是否有空闲的 Worker 可以使用,当 Worker 无法从 slaveQueue 中获取到 Event 处理时,放回到 WorkerQueue 中。

(5) Worker,对应不同的目标源有不同的 Worker,是执行线程,由从依赖图中移除的 Event 出发,并能继续从 SlaveQueue 中获取事件应用到目标端。

3.6 数据校验设计

3.6.1 概述

数据校验的目的是为了检测出在数据迁移的过程中,出现的数据不一致情况并进行响应与报警、不一致数据呈现或者进一步的数据订正。迁移过程中,数据的不一致可能出现在两个阶段:全量迁移阶段和增量迁移阶段,产生的原因包括但不限于以下几点:

- (1) 全量数据未拉取完全
- (2) 全量导入过程出错
- (3) 增量迁移的位置点发生变更
- (4) 增量迁移过程中解析 binlog 出错
- (5) 增量迁移过程中目标端 SQL 生成错误
- (6) 源端和目标端编码不同
- (7) 目标端在迁移前或者迁移过程中新增数据

有效的检测出全量和增量中产生的数据不一致,是数据校验功能的设计目标。

3.6.2 数据校验方案

(1) 设置自动校验

用户在控制台设置是否需要数据校验，以及连续校验和离散校验的比例。如果数据迁移过程中存在全量和增量两个过程，当目标端的增量数据基本追上源端时（延时 1s 内），如果用户设置了需要数据校验，则自动发起数据校验作业。而且数据校验依附于数据迁移作业，不占用 engine 资源。

(2) 主动发起校验

当迁移任务不在校验中时，用户可以在控制台主动发起校验，校验数据的全集是当前源端表中的所有数据。

(3) 主动停止校验

当迁移任务在校验中时，用户可以在控制台停止校验（停止校验不记录断点，下次发起时会重新开始）。

(4) 状态上报

上报 ValidatePhase，数据结构如表 3-16 所示，每个表的检验进度可以从 MigPipeRuntime 中获取。

表 3-16 ValidatePhase 数据结构表

validateId	long	校验 ID
inconsistentContent	int	不一致数据个数
status	JobStatus	校验状态，分别为 Init, Alive, Completed, Failed 和 Stopped
jobId	long	迁移作业 ID
sourceDbId	long	源端库 ID
startTime	long	开始时间
tableCount	int	表个数
inconsistentDatas	String	当校验结束时，返回不一致数据，格式为[源端表名 目标端表名 主键]，最多返回 100 条
endTime	long	当校验结束后，返回校验结束的时间

（5）校验历史记录

每次发起校验时，会在 `validate_history` 表中记录一条新的数据，当校验结束后会进行更新。

3.7 结构迁移设计

3.7.1 简述

假设建立迁移、同步任务，用户可以选择在目标端将表建好，然后在任务执行时如果源端表结构发生变化，则很有可能导致任务失败，所以中间件的目标是能够兼容结构迁移，包括任务开始时的源端表结构初始化以及运行过程中 DDL 的同步。对于迁移/同步任务，需要将 DDL 都同步到目标端；对于订阅任务，若源端执行了 DDL，如果为字段的增加、减少，则按照原订阅字段配置继续进行，若为订阅的字段发生 `rename`，这种情况下由于下游订阅端可能是用户自由配置，中间件的解决方案没有能够完善到同时兼顾中间件与订阅方。

这个设计实施需要考虑以下几个问题：

- （1）不同源和目标字段的适配，如 MySQL 的 `int` 对应 Oracle 的 `number` 等，生成对应目标端的建表语句等
- （2）特殊目标端的适配，如 GP，DDB 的均衡字段的问题，Kudu 不支持二级索引等问题
- （3）目标端权限检查需要检查建表、建库权限

3.7.2 全量阶段 DDL 同步

假定任务目前为全量迁移任务或者处于全量迁移阶段，源端修改了 `columnName`，任务只能手动恢复。因为根据上文描述的全量迁移方式，全量阶段过程中中间件无法感知源端产生的变化。

3.7.3 基于日志的单机 DDL 同步

首先考虑增量 DDL 出错的场景，例如原表结构为 `(id, name)`，源端删除了 `name` 列，加上了 `class` 列，`uid` 列，变成 `(id, class, uid)`。

(1) 增量迁移过程中在目标端执行 `insert into (id, class, uid) values (1,'class',1)`, 目标端由于表结构依旧为(id, class)抛 SQL 错误, 任务执行失败。

(2) 任务失败过程中, 又进行了 DDL 操作, 如由(id, class, uid) $\rightarrow$ (id, uid)。

(3) 用户发现任务失败, 修改目标表结构为(id, uid)。

(4) 用户启动任务, NDC 按照当前表结构(id,uid)去解析当前 `binlog(@1=1, @2='class',@1=1)`,binlog 解析失败, 任务再也无法恢复。

需要解决以上 DDL 带来的问题, 要点在于:

(1) 中间件需要知道某个位置点对应的表结构, 按照正常的表结构来解析 binlog

(2) 中间件需要将 DDL 操作发给下游, 由下游业务将 DDL 同步到目标端

过程模型如图 3-16 所示, 中间件将结构迁移抽象为 puller 形式提取, 并将结构迁移过程放入消息队列中, 通过并发控制结构迁移进度点与数据迁移进度点来实现任务进行过程中的结构迁移。

包含结构迁移数据落库的过程如图 3-17 所示。

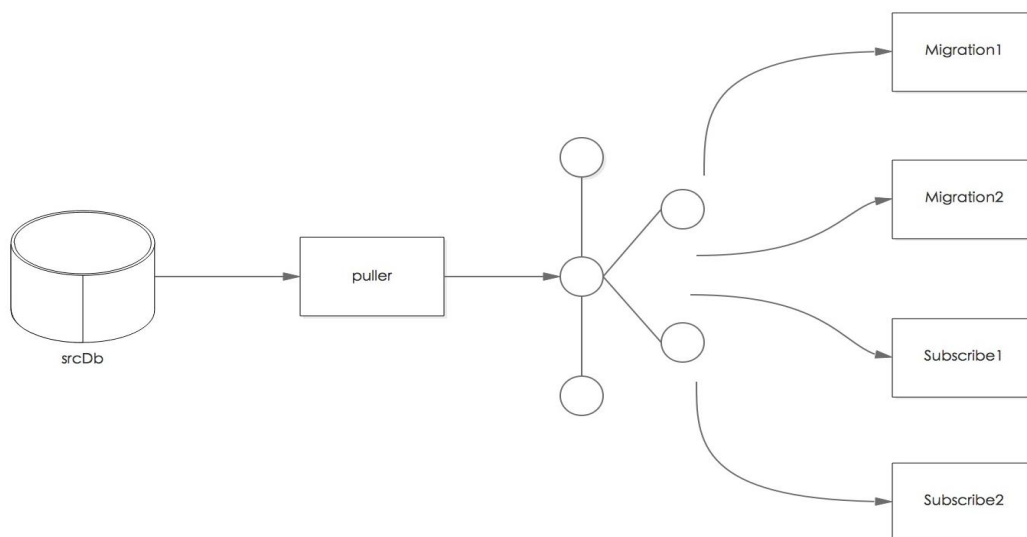


图 3-16 DDL 问题初步解决方案图

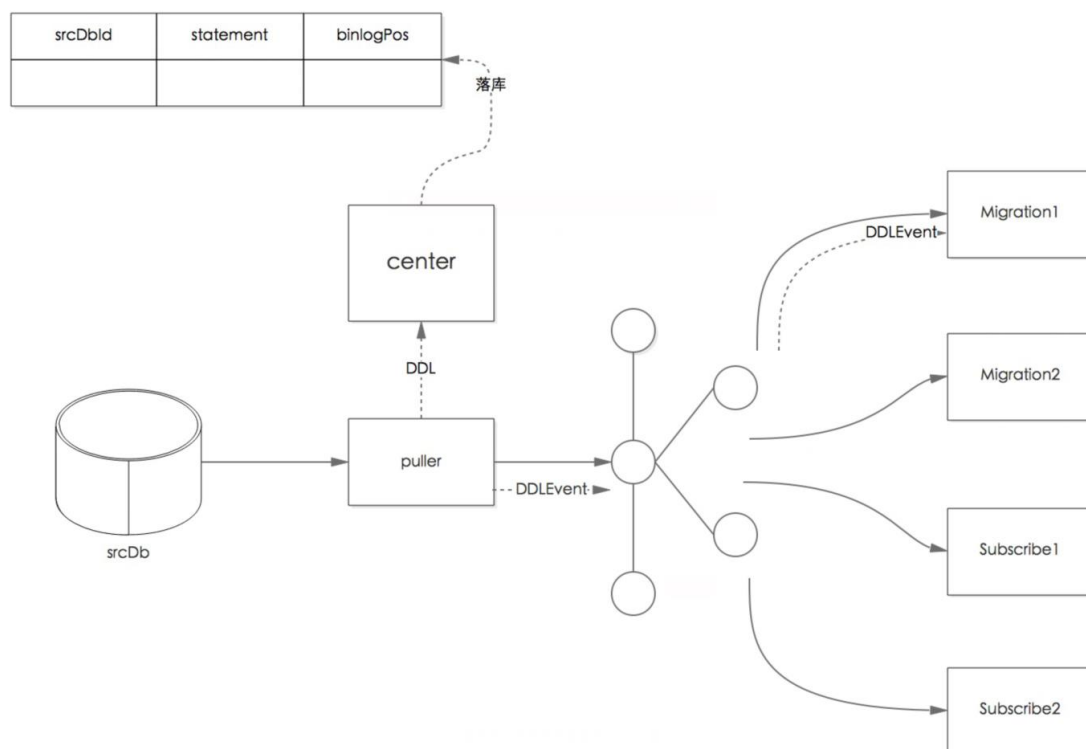


图 3-17 DDL 同步流程图

3.7.4 基于日志的分布式数据库 DDL 同步

当分布式数据库作为源的时候，任务同步包含以下特殊性：

（1）在分布式数据库为源的情况下，如果做了 DDL 操作，则会在相关的 DBN 上均执行多个 DDL

（2）一个 DDB->MySQL 的任务升级为多个 DBN->MySQL 的任务并行执行

基于以上，跟单个 MySQL 为源的增量 DDL 同步不一样，DDB 为源的 binlog DDL 除了 binlog 同步外，还需要注意以下这种情况，如图 3-18 所示。

（1）子任务 DBN2->target 已经执行到 DDL 操作，而子任务 DBN1 还未进行到 DDL 操作，还在做之前的 DML 操作。

（2）若此时让 DBN2 执行了 DDL 且同步到 targetDb，则 DBN1->targetDb 的任务将失败，且无法恢复。

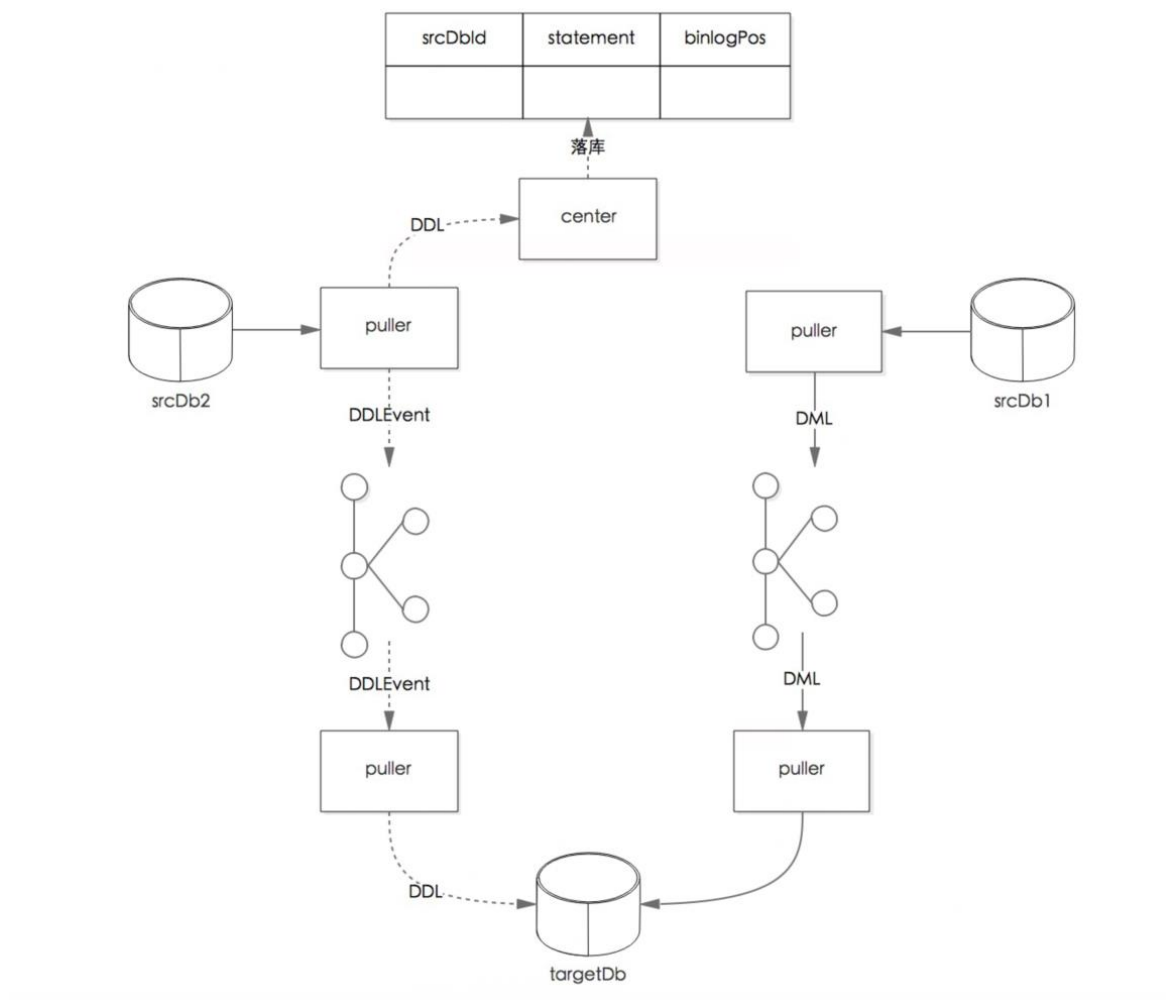


图 3-18 分布式 DDL 特殊情况图

3.8 本章小结

本章是中间件的分析与设计章节，首先从中间件需要完成的功能入手，将中间件的功能需求与性能需求提出，从而得到中间件的整体设计框图与中间件的任务模型；再将中间件主要工作模块即 Center 与 Engine 模块的设计进行简单介绍，并将性能设计与功能设计在这两个模块中体现；最后将中间件功能实现的基石模块即 Applier 模块、数据校验模块、结构迁移模块这些进行单独提取详细介绍，功能由大到小，讲解由浅入深，将整个中间件的关键设计完整呈现。

4 中间件实现

4.1 开发运行环境

中间件开发环境如表 4-1 所示。

表 4-1 中间件开发环境表

开发系统	
操作系统	Mac OS High Sierra
开发工具	IntelliJ Idea
JDK	1.8
处理器	2.3 GHz Intel Core i5
内存	16 GB 2133 MHz LPDDR3
部署环境	
操作系统	Linux
硬盘	512GB
内存	32GB

4.2 Center 控制中心实现

根据 center 的功能, 将 center 划分为以下三个模块: 节点管理 (ClusterManager)、租约管理 (SessionManager)、任务管理 (JobManager), 其它还包括数据层 dao 以及 rpc 的实现封装模块。

4.2.1 rpc 通用接口实现

Rpc 通用接口按照功能不同分为三个部分, 第一部分为展示模块, 用于外部接口或者 dashboard 获取展示信息, 主要接口如下:

```
List<String> getDatabases(DbItem dbInstance) throws DTSEException; //获取数据库信息
List<String> getTables(DbItem dbInstance, String dbName) throws DTSEException; //获取数据库表信息
List<JobRunInfo> getMigJobs(String userName) throws DTSEException; //获取迁移任务信息
List<JobRunInfo> getSubscribeJobs(String identifier, String userType, long startId, int lim
```

```
it, boolean isForward) throws DTSEException; //获取订阅任务信息
    List<EngineNode> getEngines() throws DTSEException; //获取工作节点信息
    int getMigJobTotalCount(String identifier, String userType, DataMigJob.DataMigType migType) throws DTSEException; //获取任务总量
    List<MigPipelineInfo> getMigPipelines(JobIdentifier jobIdentifier) throws DTSEException; //获取迁移表信息
```

第二部分为任务操作接口，主要包括任务的创建修改等操作；

```
long createJob(DtsJob job) throws DTSEException; //创建任务
void updateJob(DtsJob job) throws DTSEException; //更新任务
DtsJob getJob(long jobId) throws DTSEException; //获取任务信息
void startJob(long jobId) throws DTSEException; //启动任务
void suspendJob(long jobId) throws DTSEException; //暂停任务
void stopJob(long jobId) throws DTSEException; //停止任务
void releaseJob(long jobId) throws DTSEException; //释放任务
PrecheckProgress getPrecheckProgress(long jobId) throws DTSEException; //获取预检查执行过程
```

第三部分为节点管理，主要就是 session 管理等操作

```
HandshakeResponse handshake(HandshakeRequest request) throws DTSEException; //握手即心跳检测
SessionMeta register(Session session) throws DTSEException; //节点注册
GlobalConfig getGlobalConfig(String sessionId) throws DTSEException; //获取全局配置信息
boolean touch(String sessionId, GsonSerializable stat); //探活
```

4.2.2 高可用实现

高可用的实现主要封装在 HAJobController 类中，将主要接口列举如下：

```
boolean needHAHandle(JobItem jobItem, boolean isRemove); //是否需要高可用
void addJob(JobIdentifier jobIdentifier, long retryInterval, JobEvent cause); //向高可用队列添加任务
boolean jobAlreadyInHaQueue(JobIdentifier jobIdentifier, JobEvent cause); //是否已经在队列中存在
boolean isTakeOverSupportable(JobItem jobItem, JobEventType causeEventType); //是否支持任务接管
boolean takeover(JobItem jobItem, JobEventType causeEventType); //接管任务
boolean needTakeOver(JobItem jobItem, JobEvent causeEvent); //是否需要接管任务
boolean checkSrcAndTargetConnAvailable(JobItem jobItem); //检查源与目标端
```

4.2.3 实现截图

Center 控制中心实现如图 4-1 所示。



图 4-1 Center 控制中心实现图

4.3 Engine 工作节点实现

4.3.1 Engine 总体实现

Center 是中间件中的管理节点，负责发送任务相关的命令给 Engine，命令可能包括“创建任务”、“暂停任务”、“修改任务”、“取消任务”等，同时 Center 还要负责接收 Engine 上报的任务状态；而由于现在分布式架构都部署在不同的服务器上，所以它们之间的通信使用了 rpc 通信框架，rpc 通信框架的设计是为了契合系统的使用情况从而降低响应时延；任务创建后交由 Engine 中的任务管理器即 JobManager 来维护任务的生命周期；Job 下发到 Engine 后会包含多个 Process，例如预检查、全量迁移、增量迁移等，Process 有自己的 Properties 和 Status 信息，Job 维护自己 Process 的 Properties 与 JobStatus 信息的同时提供“start”、“modify”、“pause”、“resume”、“stop”等接口。Extractor 是增量复制过程中拉取源端数据的组件，Filter 是增量复制过程中对数据进行过滤的组件，Applier 是增量复制过程中将数据应用到目标端的组件。

Job 运行的完整流程如下：

(1) 预检查：数据迁移的主要任务即使源端的数据变化应用到目标端系统上，所以 engine 节点的 job 预检查暂时只包括 node 节点与源端和目标端系统的网络连通

性、账号权限、schema 校验等工作，预检查操作包括如下操作：第一首先依次执行各个预检查 process；若全部执行成功则将 job 状态修改为 READY；如果出现错误将直接抛出异常，并在用户操作页面进行提示。

(2) 运行：调用 JobExecutor 的 run 方法运行 job，只有 READY 状态下的 job 才能被运行，运行包括如下步骤：首先将 job 状态标志为 RUNNING；然后依次执行各个 Process；如果任务正常结束将状态标志为 FINISHED，若执行过程中出现错误，将状态标志为 FAILED，并挂起等待恢复。

(3) 暂停：调用 jobExecutor 的 suspend 方法暂停 job，只有 RUNNING 状态下的 job 才能被暂停，暂停操作包括如下步骤：首先将 job 状态变为 SUSPEND；紧接着调用当前正在运行的 Process 的 suspend 接口。

(4) 恢复：调用 jobExecutor 的 resume 方法恢复 job，只有 SUSPEND 和 FAILED 状态下的 job 才能被恢复，恢复操作包括如下步骤：如果 job 状态为 SUSPEND，则调用当前 Process 的 resume 接口，并将 job 状态改为 RUNNING；如果 job 状态为 FAILED，则唤醒挂起的任务线程，重试上次失败的 Process，并将 job 状态修改为 RUNNING。

(5) 停止：调用 jobExecutor 的 stop 方法停止 job，job 在任何状态下都可以被停止，停止操作包括如下步骤：首先依次调用各个 process 的 stop 接口；紧接着将任务状态修改为 STOPPED。

(6) 修改：调用 jobExecutor 的 alter 方法修改 job，job 在任何状态下都可以被修改，修改操作包括以下步骤：如果当前 jobStatus 为 RUNNING，则首先停止正在执行的 Process；然后依次调用 job 包含的 process 的 alter 接口；最后恢复 job 的执行，并从第一个 Process 开始执行。

(7) 状态上报：调用 jobExecutor 的 getJobRuntime 方法可以获得 job 的状态信息，获取状态操作包含如下步骤：首先获取当前的任务状态；然后获取当前正在执行的 process 序号；最后获取每个 process 的 jobPhase 信息。

4.3.2 Engine 全量迁移实现

在系统中，我们将任务的不同执行过程封装在名为 process 的过程中，全量迁移

任务的 process 包括了 precheckProcess 和 FullReplProcess, FullReplProcess 是负责全量复制的 Process, 我们以 MySQL 的 MySQLFullReplProcess 为例, 其批量从源端获取数据, 再将这些数据传递给根据不同目的端而确定的 FullReplApplier 中, 执行过程如下:

(1) 初始化

初始化过程中, 从 Job 对象中获取源端数据库的 ip、port、username、password 等连接信息和 batchNum、doTables 等运行时信息, 如果 runtime 不为空, 则从中获取上一次复制到的位置。

(2) 执行

任务的执行会按照 doTable 中的顺序, 依次按照 batchNum 批量获取表记录信息, 查询按照表的主键进行排序, 下次查询的主键起始值是上一次查询的最后一记录的主键取值, 用 limit 关键字控制记录的数量, 当某次获取到的数据行小于 batchNum 时表示已经获取到了表的所有数据, 使用最简单的 select 语句获取数据, 不进行任何加锁处理。获取到行数据信息后封装进 FullReplEvent 中, 并根据目的端系统的不同交由不同的 applier 将数据插入不同的目的端系统中。

全量迁移提供源端全量并行拉取、并行插入、支持断点恢复, 其中并行拉取支持两个维度: 主键分区级并行、表级并行。区别为当表的主键为 int 型时并行拉取基于主键分区, 否则在表上没有并行拉取, 任务并行拉取在该表上退化成表级并行, 关键性代码如下:

```
startApplyWorkers(); //启动数据 apply 线程, 为了保证拉取的数据都能被应用
ExtractWorker.execute(); //启动数据拉取线程
startpartitionExtractChunk(); //启动数据分区
putTaskToQueue(); //将经过上述步骤生成的任务放入待 Apply 队列
notifyExtractWorkerNoMoreTask(); //通知数据已经全部拉取完成
waitUntilFinished(); //等待直到全量任务完成
```

4.3.3 Engine 增量迁移实现

系统中，负责增量迁移的 process 命名为 ReplProcess，以 MySQL 为例，MySQLIncReplProcess 是负责 MySQL 增量复制的 Process，其模拟 MySQL 复制过程，与源端 MySQL 建立复制关系，作为从机获取源端 MySQL 的 bin log 文件存放于本地，我们将这部分 bin log 称为 relay log，MySQLIncReplProcess 再从 relay log 中读取各种事件、并过滤掉非迁移表有关的事件，再将事件封装并交给根据不同目的端确定的 applier 进行应用。

双向复制时解决数据异地多活的必须功能，双向复制必须解决的问题是数据在同步链路中一直回环。为了实现异地多活而设计的增量迁移双向复制功能基本思路为：

(1) 在需要双向复制的表中添加一指定字段，表示产生数据的数据库实例 id，如_data_src_db varchar(20)。

(2) 创建任务时，若任务为双向复制任务的子任务，则需要设定任务的源唯一标识 id，目标唯一标识 id，即为后续_data_src_db 的值。一组双向复制中，对应着两个方向的单向任务，则这两个单向任务中配置的数据库唯一 id 需要相同。

(3) 同一条数据不能在两边同时修改，中间件只能同步原写入处的更改，可能造成两边数据不一致。双向复制过程可如下图表示 NDC 解析该表的数据时候，通过_data_src_db 字段值以及任务配置的的 accepted_src_ids 来决定是否需要同步到目标端。

关键性代码如下所示：

```
tungsten.startReplication(); //tungsten 开启增量数据拉取
applier.resume(); //由 tungsten 告知 apply 模块启动
eventTransfer.transform(enent, isHamalRowEvent); //将数据进行格式转换
applier.apply(); //数据开启 apply
applyPointManager.put(); //经过 EventGraphManager 判断 Event 是否可以执行
applyFilterEvent(dataEvent); //将 Event 中数据进行过滤
waitUntilFinish(); //等待任务的中断
```

4.3.4 Engine 数据订阅实现

数据订阅根据中间件的设计主要分为三个功能，第一是序列化，第二是并行消费，第三是组订阅，实现流程在设计模块已经讲解明了，现在将主要接口展示如下：

```
tungsten.startReplication(); //tungsten 开启增量数据拉取  
applier.resume(); //由 tungsten 告知 apply 模块启动  
eventTransfer..transform(enent, isHamalRowEvent); //将数据进行格式转换  
applier.apply(); //数据开启 apply  
applyPointManager.put(); //经过 EventGraphManager 判断 Event 是否可以执行  
producer.send(record, new SendCallback(dataEvent)); //producer 将消息进行投递，且在  
send 过程中将 dataEvent 进行序列化
```

并行消费设计以及组订阅设计都是在用户使用 SDK 中设计的，主要接口如下：

```
start(ConsumerRebalanceListenr listener); //启动 SDK  
stop(); //停止 SDK  
pollEvents(long timeout); //从传输的 Records 中取出记录  
configure(Map<String, ?> configs, Boolean isKey); //设置序列化器  
serialize.deserialize(data); //将数据反序列化  
sonsumerSubscribe(); //消费者消费数据
```

4.3.5 实现截图

Engine 任务创建如图 4-2 所示。



图 4-2 任务创建图

4.4 结构迁移模块实现

4.4.1 基于日志的单机 DDL 同步实现

根据设计流程图，给出如下实现：

- (1) 遇到 DML 的 row-base binlog，puller 将 DML 解析成对应的列-值信息（DMLEvent），投放如 kafka，下游订阅到列-值信息，进行下游操作
- (2) 遇到 DDL 的 binlog，puller 解析出对应的 DDL 语句，将 DDL 以（binlog 位置点，ddl_statement，sourceId，uid）的形式发送给 center，center 将 binlog 位置点、ddl、srcId、uid 写入表 puller_src_schema 后返回
- (3) puller 将 DDL 封装成 DDLEvent，投放到 kafka，puller 更新当前 currentPos
- (4) 下游若还未消费到该 DDL，则依旧获取之间的 DMLEvent 进行操作；若下游订阅到该 DDL，则生成对对应的 DDL 操作。

任务恢复可分为两类：puller 失败，下游 apply 失败

puller 失败恢复

若 puller 失败，恢复 puller 过程为：

- (1) center 收到任务恢复请求，将系统库中该 puller 在 currentPos 之前的所有 DDL 在源库上进行重放，即 `select * from puller_src_schema where binlog <= currentPos`

(2) puller 按照重放出来的表结构, 从 currentPos 开始解析 binlog, 并将消息投递至 kafka

由于 puller 恢复时, 可能重复投递消息, 如 puller 失败时, 正在开始解析 4:DML, 而 center 中记录的 puller 的 currentPos=1 位置, 则 puller 重启恢复的时候, 会重复投递[1:DML, 2:DDL, 3:DML], 如图 4-3 所示。

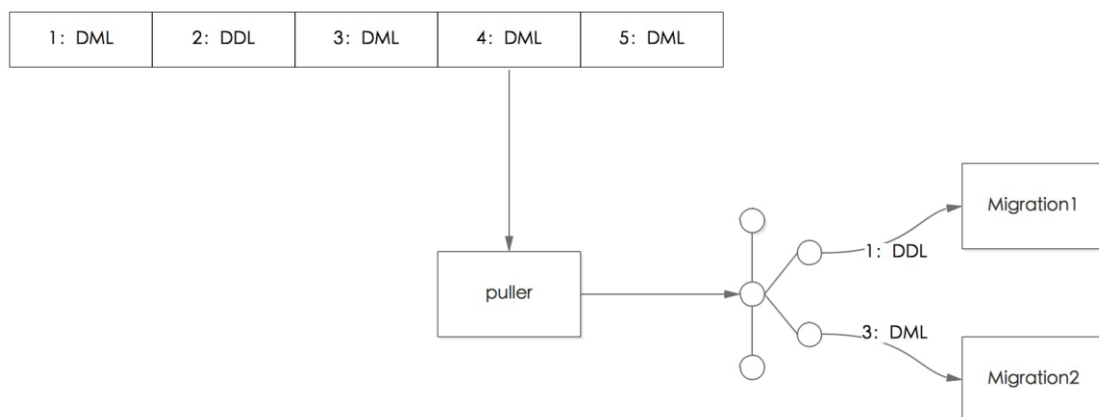


图 4-3 重复投递图

而下游有可能出于不同的进度中, 如 mig1 执行完 1:DML, 则 mig1 后续收到的时间依次为[2:DDL, 3:DML, 1:DML, 2:DDL, 3:DML]; 如 mig2 执行完 3:DML, 则 mig2 后续收到的事件依次为[1:DML, 2:DDL, 3:DML], 不论对于 mig1 还是 mig2 来说, 在第一次执行完 2:DDL 之后, 由于目标端结构已经修改, 都不应该再重复执行 1:DML。因此在下游任务处理端需要记录当前的 apply binlogPos, 当收到的消息的 binlogPos $\leq$ 当前 apply binlogPos, 则直接丢弃该消息, 否则则处理该消息。

下游 apply 恢复: 下游 apply 若失败, 恢复时从 commit offset 开始重新消费 kafka 数据, 跟前述一样, 需要通过 apply 端的 apply binlog Pos 来过滤掉已经执行过的信息, 但是由于 apply 端的 applyPos 5s 上报一次 center, 有可能 apply 端的 applyPos = 4:DML, apply 端进程意外退出, center 端保留的 apply 端的 applyPos = 1:DML, 则会导致下游 apply 重复执行[1:DML, 2:DDL]等。

因此考虑到 center 中记录的进度可能落后于 apply 实际推进的进度, 下游 apply 的恢复需要考虑两点:

(1) 正常运行中, apply 端从 kafka 中收到 Event 数据, 若 $\text{event.binlogPos} \leq \text{applyPos}$, 则跳过该 event, 否则若 event 为 DML, 则 event 数据成功 apply 后, 更新 $\text{applyPos} = \text{event.binlogPos}$; 若 event 为 DDL, 则现在目标端将该 DDL 的 pos 写入目标数据库的特定表中(可以考虑所有的操作都在目标库即 binlog Pos), 格式如 $\langle \text{pos}, \text{prepare} \rangle$, 目标库执行 DDL, 目标库特定表记录更新为 $\langle \text{pos}, \text{done} \rangle$, 更新 $\text{applyPos} = \text{event.binlogPos}$ 。

(2) apply 端重启的时候, 查看当前内存中的 applyPos 和目标端库里 binlogPos 记录, 将 applyPos 取两者最大值, 即 $\text{applyPos} = \text{Math.max}(\text{applyPos}, \langle \text{pos}, \text{prepare} \rangle | \langle \text{pos}, \text{done} \rangle)$;

```
if applyPos.type == done
    if event.binlogPos <= applyPos, 则跳过该 event;
    else      执行该 event
else      (说明不能 applyPos 对应的 DDL 是否已经执行过, 但是此 DDL 之前的数据是确定可以抛弃的)
    if event.binlogPos < applyPos, 则跳过该 event;
    else if event.binlogPos == applyPos, 执行该 event; (可能重复执行该 DDL, 需要 handle DDL 幂等问题)
    else 执行该 event
```

4.4.2 基于日志的分布式数据库 DDL 实现

对于这类分布式数据库为源的情况, 在同步 DDL 上, 需要保证所有的子任务都到达了执行 DDL 的位置点。所以实现如下:

子任务对应的 apply 执行 DDL 到目标端之前, 去 ZK 上查询, 是否所有相关子任务也达到该 DDL 位置点; 若还有其他子任务未到达该点, 则该任务出于等待状态, 暂停 apply, 否则向后推进执行。

4.4.3 实现截图

结构迁移任务创建如图 4-4 所示。



图 4-4 结构迁移选项图

4.5 Applier 模块实现

Engine 中的 Applier 模块是负责将抽取和经过过滤处理的源端数据导入到目标数据库中，在 applier 模块的执行流程如下：

(1) EventTransfer 将抽取的事件转换为 Event。

(2) 对 Event 中的记录进行处理，将主键和唯一键索引信息（用于生成导入到目标数据库的语法）记录在 Event 的 RowChange 当中，随后由 EventGraphManager 进行处理。

(3) EventGraphManager 不断从 EventQueue 中获取到 Event，根据该事件中所有涉及的主键和唯一性索引构建 EventGraph，然后放入到对应的所有的 EventGraph 中，计算其入度，如果在所有依赖图中的入度为 0，根据热定选择策略启动一个 Worker 放入线程池中执行，或者直接放入 slaveQueue 中等待其他 worker 执行。

(4) 每个 Worker 都是一个 Runnable，根据中间件设置的并发数初始化放入到一个 workerQueue 中，当 EventGraph 中存在入度为 0 的 Event。判断 WorkerQueue 是否有闲置的 Worker，如果存在取出这个 worker 绑定这个 Event 放入到线程池中执行，如果不存在就放到 salveQueue 中。

(5) Worker 的实现首先是执行出发其启动的 Event，然后循环的从 slaveQueue 中获取 Event 执行，直到没有 Event 可以处理，退出，放回到 workerQueue 中。

(6) Worker 执行完一个 Event，Worker 会通过 EventGraphManager 将该事件从所有涉及的 EventGraph 中移除，并且所有直接依赖于该事件的其余事件的入度均减一，此时会判断是否有入度为零的新事件被选出，如果是则采用上述同样逻辑进行处理，反之则等待，随后更新 ApplyPoint。

Applier 功能实现如图 4-5 所示。

NDC Dashboard

admin

数据迁移

数据订阅

订阅节点

执行节点

报警配置

用户列表

创建迁移任务

1. 源库以及目标库 2. 表选择 3. 字段选择 4. 配置选项

全量迁移

批量迁移记录数 * 200

源端拉取并发数 * 1

目标端插入并发数 * 5

增量迁移

增量并发数目 * 5

增量迁移方式 行级增量迁移

是否允许结构不一致: ☐ 是, 允许结构不一致 ☒ 否, 结构严格同步

数据校验

是否开启自动校验: ☐ 是, 自动启动校验 ☒ 否, 稍后手动启动校验

顺序校验百分比: * 50

迁移类型设置

取消 上一步 下一步 完成

图 4-5 Applier 功能实现图

4.6 数据校验模块实现

1) 校验流程如下:

(1) 记录开始时源端最新的位置点，校验的全集则为第一条记录到该位置点的所有数据，校验从源端的数据拉取和全量拉取的逻辑一致。

(2) 校验的过程中，如果出现源端和目标端的数据不一致，则将不一致的主键记录在内存中

(3) 启动定时线程，定期检查不一致的数据集合，当两边数据一致时，将该主

键从集合中移除

(4) 当存在连续校验的比例非 100%时, 假设为 50%, 对于每次校验的一批数据, 前 50%的数据为连续校验, 后 50%的数据为抽样校验。

2) 结束, 结束情况主要分为以下几种:

(1) 数据校验完成后, 不一致数据集合为空, 结果为校验成功。

(2) 数据校验的过程中, 若不一致数据量超过一定值, 结果为校验失败。(如果不一致数据量一致保持在这个值以内, 校验会一致进行下去直到遇到其他的结束条件)

(3) 校验过程抛出异常

(4) 用户主动停止校验

3) Center 主动获取不一致数据列表

center 可以主动调用 Engine 的 RPC 接口, `List<InconsistentRecord> getInconsistentDatas (JobIdentifier identifier)` 来获取不一致数据。

数据校验结果显示如图 4-6 所示。



图 4-6 数据校验结果图

4.7 数据迁移速度测试

中间件全量迁移与增量迁移速度是中间件满足市场需求的重要保证，现就中间件的数据迁移速度与传统数据迁移方式以及国内市场上先进的工具进行对比，测试得出对比结果如图 4-7 所示。

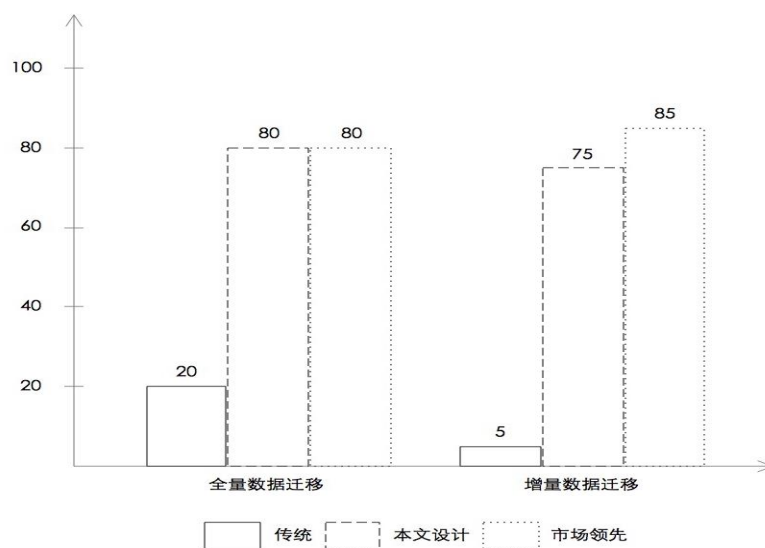


图 4-7 测试对比图

由于数据敏感性问题，只是列出了三者之间的数据对比，从图中可以看出本文设计的中间件全量迁移速率是传统数据迁移方式的 4 倍且已经是目前市场上的领先水平，增量迁移速度更是传统迁移方式的 15 倍左右且也已经接近市场领先水平，充分体验了本文中间件设计实现的价值。

4.8 本章小结

本章主要根据第三章的设计思想将中间件进行逐层分析实现，首先介绍了中间件的开发环境，随后按照第三章设计的思路由大到小逐步讲解中间件中功能设计与性能设计的实现，内容主要包括对设计流程的介绍与实现、功能接口以及功能实现截图，最后将中间件的数据迁移速度进行测试并对比传统数据迁移方式与国内市场领先的数据迁移工具，更加体现了本文设计并实现的中间件的价值。

5 总结与展望

5.1 全文总结

基于上述的几章讲解，本文先从数据迁移中间件的开发背景讲起，分析了当下快速增长的海量互联网数据对于中间件提出的类似不停服数据迁移、数据异地备灾、异地多活以及提供线上 OLAP 系统新的解决方案等需求。

紧接着讲述了在中间件中采用的一些关键技术，如 Zookeeper 主要用于中间件中 center 与 engine 的心跳检测、engine 工作汇报等；如 Kafka 在大的功能模块上提供数据订阅的功能，在小的功能模块上为一推多拉、任务结构迁移提供更优选择的解决方案；如 Rpc 通信用于在 dashBoard 与 center 的数据通信以及 center 与 engine 的数据通信来降低中间件的响应延迟提升用户的使用体验等。

第三章通过功能需求分析与性能需求分析两个方面初步讲解了中间件需要具备的能力以及为什么需要设计这些能力。紧接着就中间件的各个设计模块进行了详细的设计与实现，为了使读者能够更容易理解，讲解的顺序也是以任务流程顺序为基础的，首先从 center 控制中心的设计，主要讲解了 center 基本的 API 以及任务下发流程的优化；其次就来到了 engine 执行节点的设计，这部分包含的内容比较多，首先对 engine 的专有名词进行了说明，再从 engine 最上层调用流程为切入点，进而结合 engine 的常用 API 为大家介绍了 engine 内部的组件以及它们的功能；进而更加深入地介绍中间件主要功能“全量迁移”与“增量迁移”的实现细则。紧接着将这两个主要功能中的三个公有模块“结构迁移”、“Applier”与“数据校验”提取出进行单独讲解，三个章节都从设计方案、执行流程与细节方面对功能实现进行了进一步诠释，全文在讲解完主要功能后进入尾声，每一步设计的目的都是为了功能更好地实现以及用户更好的体验。

5.2 展望

在讲解完中间件的中体功能与设计实现后，确实中间件还存在许多值得商榷的设计点。

比如在一拉多推的设计上，虽然能够减少链路途径，减少网络消耗导致的延时以及源端的压力，但是缺点也很明显。首先必须依赖 Kafka，这样的话 Kafka 里 topic 数量过多可能会被 DBA 的运维造成很大的压力，而且不能保证表的顺序，虽然对于数据迁移不会产生影响但是对订阅可能会产生影响，以同一源新建的任务都必须投递到统一 Kafka 否则就需要 puller 投递多个 kafka，这些都是新的架构带来的问题。

如上文提到的在结构迁移上存在全量迁移无法被感知的问题，这个问题的解决就要从最初的全量迁移架构的设计上去修改，对中间件的破坏性太大，同样需要去思考更加全面的方式去优化解决。

系统的设计没有最完美的方式，只有更加契合需求的实现方式，所以随着互联网的发展，需求的更新也必然会推动系统的改进，希望通过这个系统能解决更多用户在数据迁移方面的问题，为用户提供稳定、安全、快速的服务。

致 谢

系统的设计为了尽可能能满足用户的需求，需要从设计到实现方方面面的考量，非常感谢网易公司能够给予我这次宝贵的机会参与到这个中间件的设计与实现中。能得到这次实习的机会也非常感谢方老师以及同学们在研一期间对学习环境的提供、学习方向上的引导以及学习方法上的传授。

非常感谢方老师在论文完成过程中的指导，包括论文主体结构的设计，论文亮点的凸显，以及论文后期的增补修改，方老师以她个人温淳的性格、渊博的学识、严谨的学术态度影响着我们，让我们在学校期间生活能够更加方便、学习能够更加稳重踏实，也教会了我们许多做人的道理，让即将踏入社会的我们不那么迷茫，对此我对方老师表达我最诚挚的感谢与祝福。

同时也要感谢学院与学校给予我这短暂且令人倍加珍惜的学习机会，华科浓郁的学术氛围，优良的学习生活环境都对我这三年的提升与成长起到了非常大的助力作用，华科明德、厚学、求是、创新的校训也在一直不停地激励着我们砥砺前行，最后要祝愿软件学院与华科能够越来越好、人才济济桃李满天下！

即将离开校园的我们也会一直惦念着母校，不忘母校的教导，在生活与科研上披荆斩棘、长风破浪做一个无愧于母校与国家有用的人！

参考文献

- [1] 张联梅, 王和平. 软件中间件技术现状及发展. 信息通信, 2018(5): 66-69
- [2] 王丹丹, 陈康. 软件工程技术现状与发展趋势. 信息与电脑(理论版), 2016(6): 50-51
- [3] 戴德宝, 范体军, 刘小涛. 互联网技术发展当前中国经济发展互动效能分析. 中国软科学, 2016(8): 184-192
- [4] 冯艳茹. 分享经济——互联网技术发展背景下的商业新模式. 时代金融, 2016(17): 24-25
- [5] 徐燕, 梁正和. 信息系统中的通用数据迁移工具的研究与设计. 计算机与现代化, 2010(6): 156-158
- [6] 刘智慧, 张泉灵. 大数据技术研究综述. 浙江大学学报: 工学版, 2014, 48(6): 957-972
- [7] Forite L, Hug C. FASMM: Fast and Accessible Software Migration Method. IEEE Eighth International Conference on Research Challenges in Information Science. IEEE, 2014: 33-35
- [8] Zhao Z, Yao H. A Data Backup Method Based on File System Filter Driver, 2010: 33-37
- [9] Lee H C, Park J E, Lee M J. C3ware: A Middleware Supporting Collaborative Services over Cloud Storage. The Computer Journal, 2014, 57(2): 217-224
- [10] 刘鹏远. 异步消息机制原理挖掘与应用范畴分析. 计算机系统应用, 2014, 23(8): 203-207
- [11] 张棣, 曹健. 面向大数据分析的决策树算法. 计算机科学, 2016, 43(Z6): 155-160
- [12] Shvachko K, Kuang H, Radia S, et al. The Hadoop Distributed File System. IEEE Symposium on Mass Storage Systems & Technologies. IEEE Computer Society, 2010: 124-132
- [13] Florescu D, Kossmann D, Manolescu I. Integrating keyword search into XML

- query processing. *Computer Networks*, 2000, 33(1-6): 119-135
- [14] M Bouzeghoub, M Lenzerini. Introduction to: Data Extraction, Cleaning, and Reconciliation. *Information Systems*, 2001, 26(8): 535-536
- [15] Tseng F S C, Hwung W J. An automatic load/extract scheme for XML documents through object-relational repositories. *Journal of Systems and Software*, 2002, 64(3): 207-218
- [16] Williams M E, Consolazio G R, Hoit M I. Data storage and extraction in engineering software using XML. *Advances in Engineering Software*, 2005, 36(11-12): 709-719
- [17] Marks R M. A Metadata Driven Approach to Performing Multi-vendor Database Schema Upgrades. *IEEE International Conference & Workshops on Engineering of Computer Based Systems*. IEEE, 2012: 477-482
- [18] Wang L, Rundensteiner E A, Mani M. Updating XML views published over relational databases: Towards the existence of a correct update mapping. *Data & Knowledge Engineering*, 2006, 58(3): 263-298
- [19] Fenk R, Kawakami A, Markl V, et al. Bulk loading a Data Warehouse built upon a UB-Tree. *International Symposium on Database Engineering & Applications*. IEEE Computer Society, 2000, 880576
- [20] Leverich J, Kozyrakis C. On the energy(in)efficiency of Hadoop clusters. *Acm Sigops Operating Systems Review*, 2010, 44(1): 61-65
- [21] Song M, Luo G, Haihong E. A Service Discovery System based on Zookeeper with Priority Load Balance Strategy. *IEEE International Conference on Network Infrastructure & Digital Content*. IEEE, 2017: 252-255
- [22] Carcillo F, Pozzolo A D, Borgne Y A L, et al. SCARFF: a Scalable Framework for Streaming Credit Card Fraud Detection with Spark. *Information Fusion*, 2017: S1566253517305444
- [23] An K, Gokhale A, Tambe S, et al. Proceedings of the Posters and Demos Session of the 16th International Middleware Conference on - Middleware Posters and Demos '15 - Wide Area Network-scale Discovery and Data Dissemination in Data-centric Publish/Subscribe Systems, 2015: 1-2
- [24] Vora M N. Hadoop-HBase for large-scale data. *International Conference on*

- Computer Science & Network Technology. IEEE, 2012: 455-462
- [25] 杨宁, 黄婷婷. 基于 Spark 的大数据分析工具 Hive 的研究. 计算机时代, 2018(11): 31-35
- [26] Carcillo F, Pozzolo A D, Borgne Y A L, et al. SCARFF: a Scalable Framework for Streaming Credit Card Fraud Detection with Spark. Information Fusion, 2017: S1566253517305444
- [27] 李东辉, 吴小志, 朱广新, 等. 分布式数据库协调技术-Zookeeper. 科技展望, 2016, 26(1): 212-215
- [28] Han-ShengChuang, Liang-TehLee, Che-YuanChang, et al. Modified Packing Algorithm for Dynamic Energy-Savingin Cloud Computing Servers. 电子科技大学, 2013, 11(2): 124-131
- [29] 冉勇, 曹东升, 杨婧孜, 等. 基于 DUBBO+ZOOKEEPER 计量服务平台研究. 计量与测试技术, 2017(1): 41-42
- [30] 杨冬晖. 一种分布式消息队列的可靠性研究. 电脑知识与技术, 2015(21): 75-76
- [31] Wang Z, Dai W, Wang F, et al. Kafka and Its Using in High-throughput and Reliable Message Distribution. International Conference on Intelligent Networks & Intelligent Systems. IEEE, 2016: 25-27
- [32] Esmaili K S, Esmaili K S. Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations: Industry Paper. Acm International Conference on Distributed & Event-based Systems, 2017: 155-163
- [33] 张艳军, 王剑, 叶晓平. 基于 Netty 框架的高性能 RPC 通信系统的设计与实现. 工业控制计算机, 2016, 29(5): 11-12
- [34] Shvachko K, Kuang H, Radia S, et al. The Hadoop Distributed File System, 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies(MSST). IEEE, 2010, 277
- [35] 柯研, 马凯, 郑钰辉. 分布式计算平台 Hadoop. 数字技术与应用, 2018, 36(9): 70-71
- [36] 徐涛. 结构化大数据存储与查询优化关键技术: [博士学位论文]. 北京: 清华大学图书馆, 2016

- [37] Wang F, Qiu J, Yang J, et al. Hadoop high availability through metadata replication. DBLP, 2009: 255-259
- [38] Taylor R C. An overview of the Hadoop/MapReduce/HBase framework and its current applications in bioinformatics. BMC Bioinformatics, 2010, 11 Suppl 12(S12): S1
- [39] Agneeswaran V S. Big data analytics beyond Hadoop real-time applications with Storm, Spark, and more Hadoop alternatives, 2014: 267-274
- [40] Aji A, Wang F, Vo H, et al. Hadoop-GIS: A High Performance Spatial Data Warehousing System over MapReduce. Proceedings of the VLDB Endowment, 2013, 6(11): 1009-1020